Lab 3 – 7-Segment Display Controller

Design, verification and implementation on the Digilent Nexys A7-100T (Artix-7 xc7a100tcsg324-1)

Course: Chip Design and Verification – Verilog + Verification

Student: Yuval

Tools: Vivado 2025.2 (xsim, synthesis, implementation), Verilator 5.052 (extra lint)

Date: October 2026

Contents

Contents . . . 2

Summary of results . . . 3

1. Objective and requirements **3**

2. Design **3**

2.1 Architecture . . . 3

2.2 Timing design: refresh rate, 1-second hold and the 1 Hz question . . . 4

2.3 Module details . . . 4

2.4 Board interface and polarities . . . 5

2.5 Design process . . . 5

3. Verification – behavioral simulation **6**

3.1 Method and results . . . 6

3.2 tb_ss – seven-segment decoder . . . 6

3.3 tb_tick_gen – clock divider . . . 9

3.4 tb_value_latch – switch value register . . . 10

3.5 tb_btn_hold_1s – 1-second hold detector . . . 12

3.6 tb_binary_2_bcd – double-dabble converter . . . 16

3.7 tb_display_ctrl – digit scanning . . . 19

3.8 tb_chip_top – full system . . . 23

4. RTL analysis – lint and elaborated design **28**

4.1 Lint . . . 28

4.2 Elaborated design checks . . . 29

5. Synthesis **32**

6. Implementation **33**

7. Resource expectations and usage **35**

7.1 Stage 1 – expectations from the RTL (filled in before looking at the reports) . . . 35

7.2 Stage 2 – usage measured from the reports . . . 35

7.3 Do the used resources meet the expectations? . . . 36

8. Bitstream and board test **36**

9. Conclusions **36**

Appendix A – Deliverables **36**

Appendix B – Source listing **37**

Summary of results

Item	Result
Design	7 Verilog modules, single 100 MHz clock domain, 91 flip-flops / 80 LUTs after implementation
Simulation	7 self-checking testbenches, 798 checks, 0 failures (xsim, timescale 1ns/1ps)
Lint	Vivado linter: 0 violations for chip_top and every sub-module; Verilator -Wall: 0 errors, 0 warnings on the 6 sub-modules, 2 style notes on chip_top
Elaborated design	Report DRC: 0 checks, Report Methodology: 0 checks
Synthesis	0 errors, 0 critical warnings, 4 benign warnings (ports tied to constant 1: DP, AN7..AN5)
Implementation	0 errors / 0 critical / 0 warnings; DRC clean; 100 % routed; timing met
Timing @ 100 MHz	setup WNS +5.326 ns, hold WHS +0.163 ns, pulse width +4.5 ns (0 failing endpoints of 237)
Resources	80 slice LUTs (0.13 %), 95 FF (0.07 %), 35 slices (0.22 %), 36 IOB (17.1 %), 0 BRAM/DSP (from the utilization .rpt files)
Bitstream	chip_top.bit generated with 0 warnings; programmed and tested on the Nexys A7-100T

1. Objective and requirements

The goal of the lab is to design and implement a controller for the 8-digit common-anode 7-segment display of the Nexys A7 that shows the 16-bit value set on the slide switches SW[15:0], either in decimal (up to 5 digits, 65535) or in hexadecimal (4 digits, FFFF). The value is not shown live: it is latched into a register when the center push-button (decimal) or the down push-button (hexadecimal) is held for 1 second. The CPU reset button (active-low) clears the register and selects hexadecimal 0. The display must not flicker, so every digit is refreshed at 400 Hz from a divided 100 MHz system clock.

- Inputs: CLK100MHZ, SW[15:0], BTNC (center), BTND (down), CPU_RESETN (active-low).
- Outputs: disp_an[7:0] – digit anodes (active-low), ss_dig[6:0] = {CA..CG} and DP – cathodes (active-low).
- Switch value latching with a 1 s hold, decimal / hex presentation, reset to 0.
- Binary-to-BCD conversion for the decimal presentation; a 7-segment decoder per digit value.
- Testbench per building block plus a system testbench, push-button time reduced to ~100 ns in simulation.
- Physical implementation on the board: XDC pin assignment, synthesis, place and route, bitstream, board test.

2. Design

2.1 Architecture

The design is a single-clock synchronous design: everything runs on the 100 MHz board clock and all slower events are one-clock-cycle enable pulses (ticks), never derived clocks. Figure 1 shows the elaborated top level: chip_top synchronizes the reset, instantiates one tick generator, two identical button-hold detectors, the value latch, the binary-to-BCD converter and the display controller (which contains the 7-segment decoder).

Module	File	Function	Parameters (hardware value)
chip_top	src/chip_top.v	Top level: 2-FF synchronizer for CPU_RESETN, wiring of all blocks, start = pressed_c | pressed_d	MAX_TICK = 50000, MAX_COUNT = 2000
tick_gen	src/tick_gen.v	Divides the clock into a one-cycle tick: counter 0..MAX_TICK-1, tick when the counter reaches MAX_TICK-1 (synchronous clear)	MAX_TICK = 50000 → 2 kHz tick
btn_hold_1s	src/btn_hold_1s.v	2-FF synchronizer on the asynchronous button; counts ticks while the button is held; one-cycle pulse pressed_1s when MAX_COUNT ticks were counted, then saturates (one pulse per press)	MAX_COUNT = 2000 ticks = 1.000 s
value_latch	src/value_latch.v	sw_reg <= SW on pressed_c or pressed_d; dec_hex <= 1 on center, 0 on down (down has priority when both); reset → 0 / hex	–
binary_2_bcd	src/binary_2_bcd.v	Double-dabble (shift-and-add-3) converter: start clears, then 16 shift cycles produce 5 BCD digits (160 ns)	–

Module	File	Function	Parameters (hardware value)
display_ctrl	src/display_ctrl.v	5-bit one-hot ring (active-low) rotated on every tick selects the anode and the nibble; 5th anode enabled only in decimal mode; DP always off	–
seven_segment	src/seven_segment.v	Combinational hex digit → 7 active-low cathodes {CA,CB,CC,CD,CE,CF,CG}	–

2.2 Timing design: refresh rate, 1-second hold and the 1 Hz question

The 100 MHz clock is divided once, by tick_gen, into a 2 kHz tick (MAX_TICK = 50000 → 100 MHz / 50000 = 2000 Hz, i.e. one tick every 0.5 ms). The display controller advances one digit slot per tick. There are 5 slots (AN0..AN4), so every digit is lit once per 5 × 0.5 ms = 2.5 ms, i.e. a refresh rate of exactly 400 Hz per digit, as required. In hexadecimal mode the 5th slot keeps its place but all anodes are off, so the refresh period and the brightness are identical in both modes.

Lab question – is it feasible to create a 1 Hz pulse out of the system clock, and can we use another pulse / divided clock? A 1 Hz pulse straight from 100 MHz is feasible but needs a 27-bit counter (100,000,000 cycles). Instead, the button-hold detector re-uses the 2 kHz display tick as its time base and counts 2000 ticks with an 11-bit counter (MAX_COUNT = 2000 → 2000 × 0.5 ms = 1.000 s). This saves 16 flip-flops and 4 carry chains per button and, more importantly, keeps one clock domain: the tick is a synchronous enable, so no clock-domain crossing, no extra BUFG and no timing-analysis issues are introduced. A divided clock used as a real clock would have required all of those.

Both parameters default to the hardware values inside the modules; the testbenches only override them at instantiation (tb_chip_top: MAX_TICK = 2, MAX_COUNT = 5, so one '1 s' press is 10 clocks = 100 ns). Nothing has to be changed back when targeting the real board.

2.3 Module details

tick_gen

A free-running counter of ⌈log2(MAX_TICK)⌉ = 16 bits. tick = (counter == MAX_TICK-1) is combinational and is also the synchronous clear of the counter, so the tick is exactly one clock wide every MAX_TICK clocks (Figure 11).

btn_hold_1s

The button is an asynchronous input, so it first passes a 2-flip-flop synchronizer (click_sync0, click_sync). An 11-bit counter_1s is cleared whenever the synchronized button is released (or on reset) and increments on every tick while it is held; it saturates at MAX_COUNT. The output pressed_1s = click_sync & tick & (counter_1s == MAX_COUNT-1) is therefore a single one-clock pulse exactly when the 2000th tick arrives – holding the button longer does not generate a second pulse, and releasing earlier generates none (Figures 15–18). Contact bounce shorter than one tick is ignored because the counter is cleared on every release.

value_latch

Two registers: sw_reg[15:0] loads SW when either pulse arrives; dec_hex is set by the center pulse and cleared by the down pulse or reset. If both buttons produce a pulse in the same cycle, the down (hex) branch is evaluated first and wins (Figures 13–14). After reset the display shows 0 in hexadecimal mode.

binary_2_bcd

Double-dabble algorithm implemented with a 5-bit cycle counter and five 4-bit digit registers. start (= pressed_c | pressed_d) clears everything in the same cycle sw_reg is loaded; then for 16 cycles the next MSB of sw_reg is shifted in while every digit ≥ 5 has 3 added before the shift (the ten-thousands digit can never exceed 6, so it needs no add-3). After 16 shifts the counter stops at 16 and the result is held (Figures 19–21). The conversion takes 160 ns, far below the 0.5 ms tick, so the display never shows a partial result.

display_ctrl and seven_segment

ring_reg[4:0] resets to 11110 and rotates left on every tick, giving an active-low one-hot anode pattern. display_an = {111, !dec_hex | ring[4], ring[3:0]}: anodes 7..5 are always off, anode 4 is only driven in decimal mode. The nibble sent to the decoder is selected from val_to_show = dec_hex ? bcd_val : {0000, sw_reg}. The

seven_segment decoder is a 16-entry case statement producing the active-low cathode pattern for 0-9 and A-F; the decimal point is tied off (DP = 1).

2.4 Board interface and polarities

Signal	Direction	Polarity on the Nexys A7	Pins (XDC)
CLK100MHZ	in	100 MHz oscillator, create_clock 10 ns (sys_clk_pin)	E3
CPU_RESETN	in	0 when the red CPU RESET button is pressed	C12
BTNC / BTND	in	1 while pressed	N17 / P18
SW[15:0]	in	1 when the switch is up	J15 L16 M13 R15 R17 T18 U18 R13 T8 U8 R16 T13 H6 U12 U11 V10 (SW8, SW9 are LVCMOS18)
disp_an[7:0]	out	0 turns the digit on (common anode via PNP driver); AN0 is the rightmost digit	J17 J18 T9 J14 P14 T14 K2 U13
ss_dig[6:0] = {CA..CG}	out	0 lights the segment	T10 R10 K16 K13 P15 T11 L18
DP	out	0 lights the decimal point – always 1 (off) in this design	H15

2.5 Design process

The controller was built bottom-up. Each block was written in its own file and immediately given a self-checking testbench (the testbenches compare the DUT against a behavioral model written in the testbench, for example ss2dig() decodes the cathode pattern back into a digit and bin2bcd() computes the expected BCD value with division). Order of development: seven_segment → tick_gen → display_ctrl → btn_hold_1s → value_latch → binary_2_bcd → chip_top. The Vivado flow (linter, elaborated design with schematics, synthesis, implementation) was run on the individual modules as they were finished, then on chip_top; the bitstream was generated last and tested on the board. Before packaging, every testbench and the whole Vivado flow were re-run on the final sources so that all reports in this document describe exactly the delivered code. Every design and testbench file starts with `timescale 1ns/1ps, as the lab requires (the simulations and the Verilator lint were re-run after adding it to the files that lacked it).

One bug found by simulation is worth mentioning: tb_binary_2_bcd originally declared the DUT output as reg, which xsim rejects ('concurrent assignment to a non-net'); it was changed to wire. tb_display_ctrl also caught the need to turn anode 4 off immediately when switching from decimal to hex in the middle of a scan, which is why the anode-4 enable is combinational (!dec_hex | ring[4]) rather than registered.

3. Verification – behavioral simulation

3.1 Method and results

All testbenches run in Vivado xsim with timescale 1ns/1ps and a 10 ns clock. Every testbench drives stimulus, predicts the expected result in Verilog, prints one PASS/FAIL line per check and ends with a TEST PASS/FAIL summary and $finish. The push-button hold time is shortened to 100 ns in simulation by parameter overrides only. The waveforms below were rendered from the xsim VCD dumps; the numbered flags are markers placed on the important events and the shaded bands name the test sections. The red dashed boxes in an overview mark where the following zoom-in figures are taken.

Testbench	DUT	What is checked	Checks	Result
tb_ss	seven_segment	all 16 digits: cathode pattern decoded back equals the input	16/16	PASS
tb_tick_gen	tick_gen (MAX_TICK=5)	every tick interval = MAX_TICK clock cycles, tick is 1 clock wide	19/19	PASS
tb_value_latch	value_latch	reset values, no latch without press, center/down latch and mode, hold, both pressed	6/6	PASS
tb_btn_hold_1s	btn_hold_1s (MAX_COUNT=2)	short presses give no pulse, a hold gives exactly one pulse after MAX_COUNT ticks (latency window checked), all tick phases, bounce	18/18	PASS
tb_binary_2_bcd	binary_2_bcd	start clears; 0, 65535, 12345, 67890 (wraps to 2354) and 20 random values compared with a division model	25/25	PASS
tb_display_ctrl	display_ctrl + seven_segment	anode sequence per tick in hex (AN4 never on) and decimal (AN4 on), digit on the cathodes at every position, mode switch mid-scan, value change mid-scan, 40 random values, DP off	525/525	PASS
tb_chip_top	chip_top (MAX_TICK=2, MAX_COUNT=5)	system: what each anode shows after every scenario – reset, 12345 dec, hold without press, 777 hex, short press ignored, 65535/FFFF, 0, ABCD both buttons, 4321, reset, 10 random values	189/189	PASS

Total: 798 checks, 0 failures. Logs: 1_Simulation_Waveforms/simulation_logs/<tb>_xsim.log.

3.2 tb_ss – seven-segment decoder

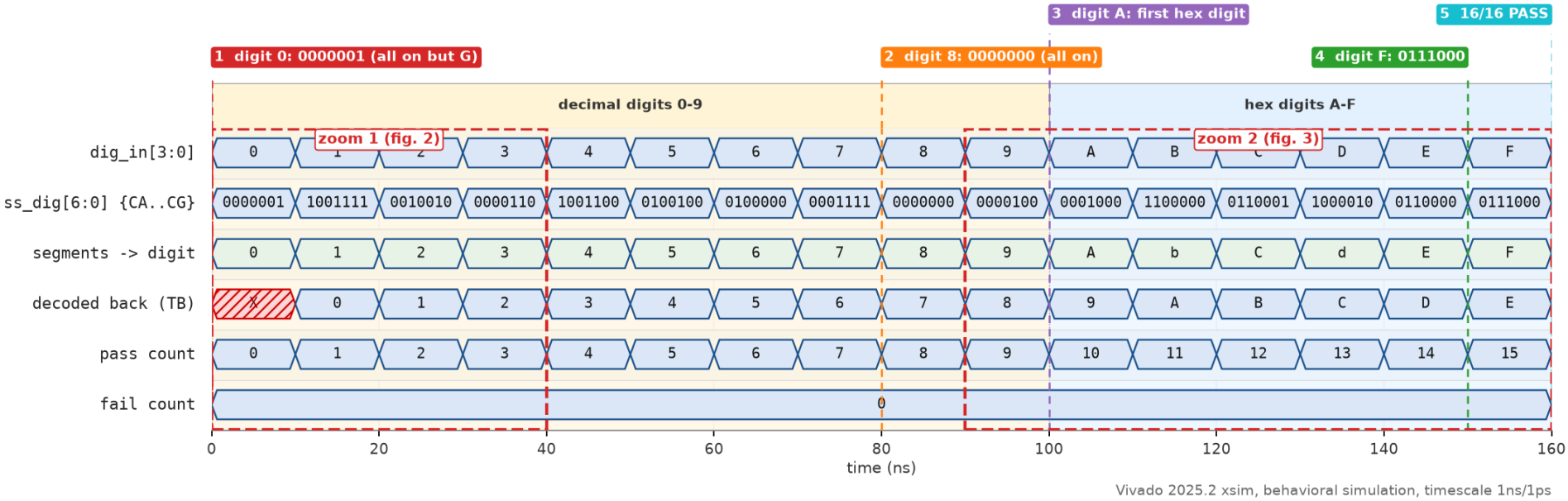


Figure 1: tb_ss overview: dig_in counts 0..F every 10 ns; ss_dig shows the active-low pattern; 'segments → digit' decodes it back. Markers 1–4 point at 0, 8 (all segments on), A (first hex digit) and F.

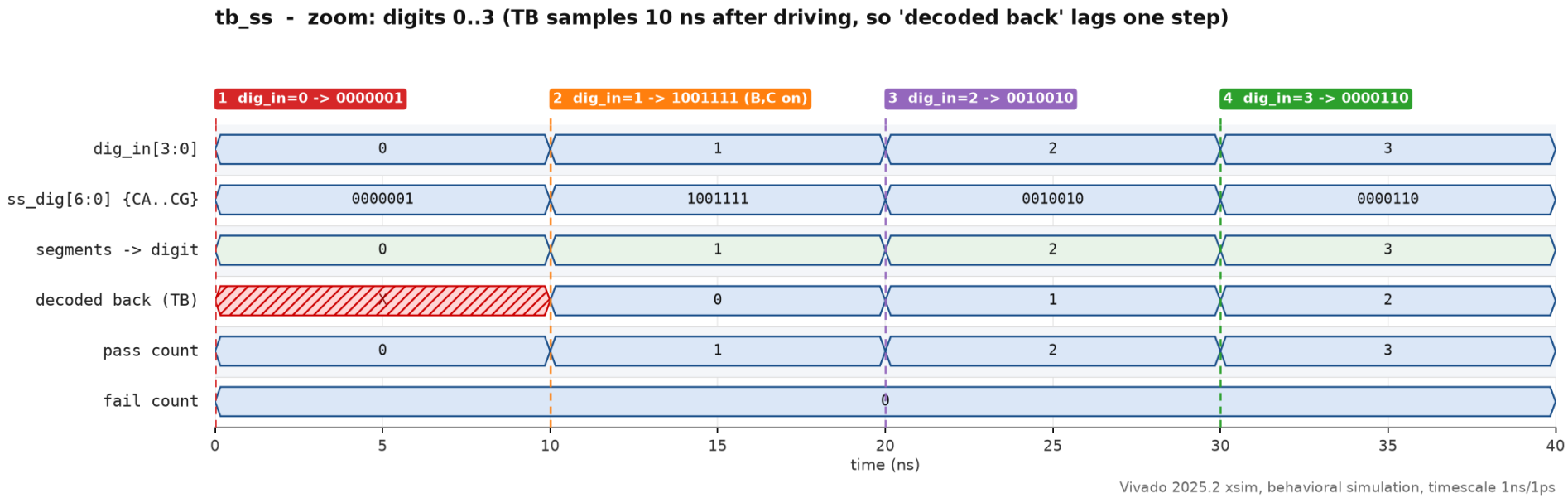


Figure 2: Zoom on digits 0..3: 0 = 0000001 (only G off), 1 = 1001111 (B, C on), 2 = 0010010, 3 = 0000110. The testbench samples 10 ns after driving, so its 'decoded back' value lags one step.

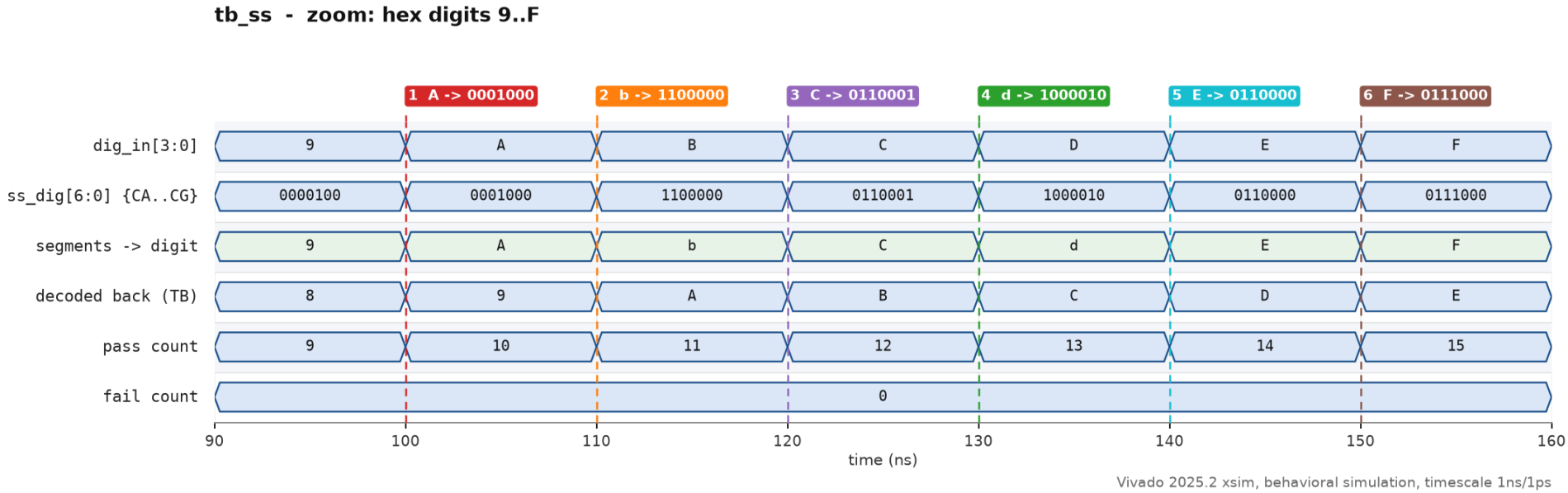


Figure 3: Zoom on the hex digits A..F (lower-case b and d patterns distinguish them from 8 and 0).

3.3 tb_tick_gen – clock divider

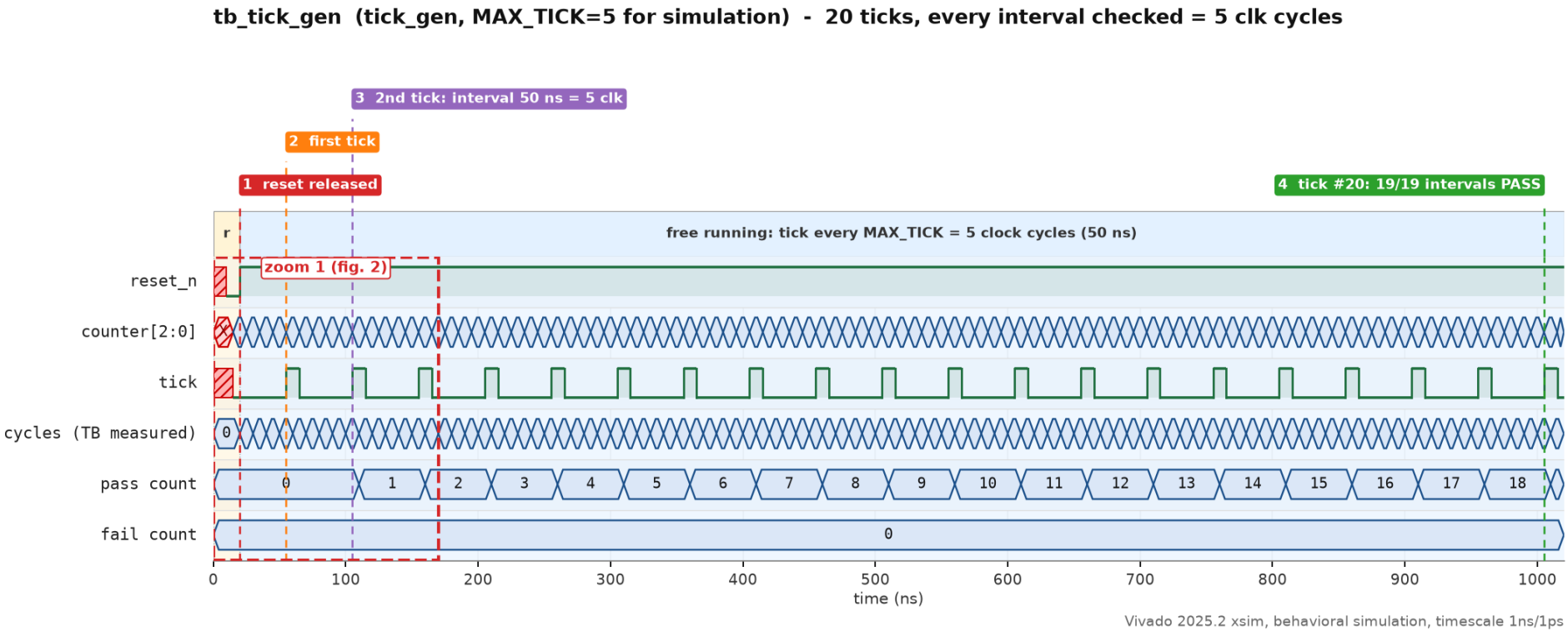


Figure 4: tb_tick_gen overview with MAX_TICK = 5: after reset the tick appears every 50 ns (5 clocks); the testbench measures 19 consecutive intervals, all equal to 5.

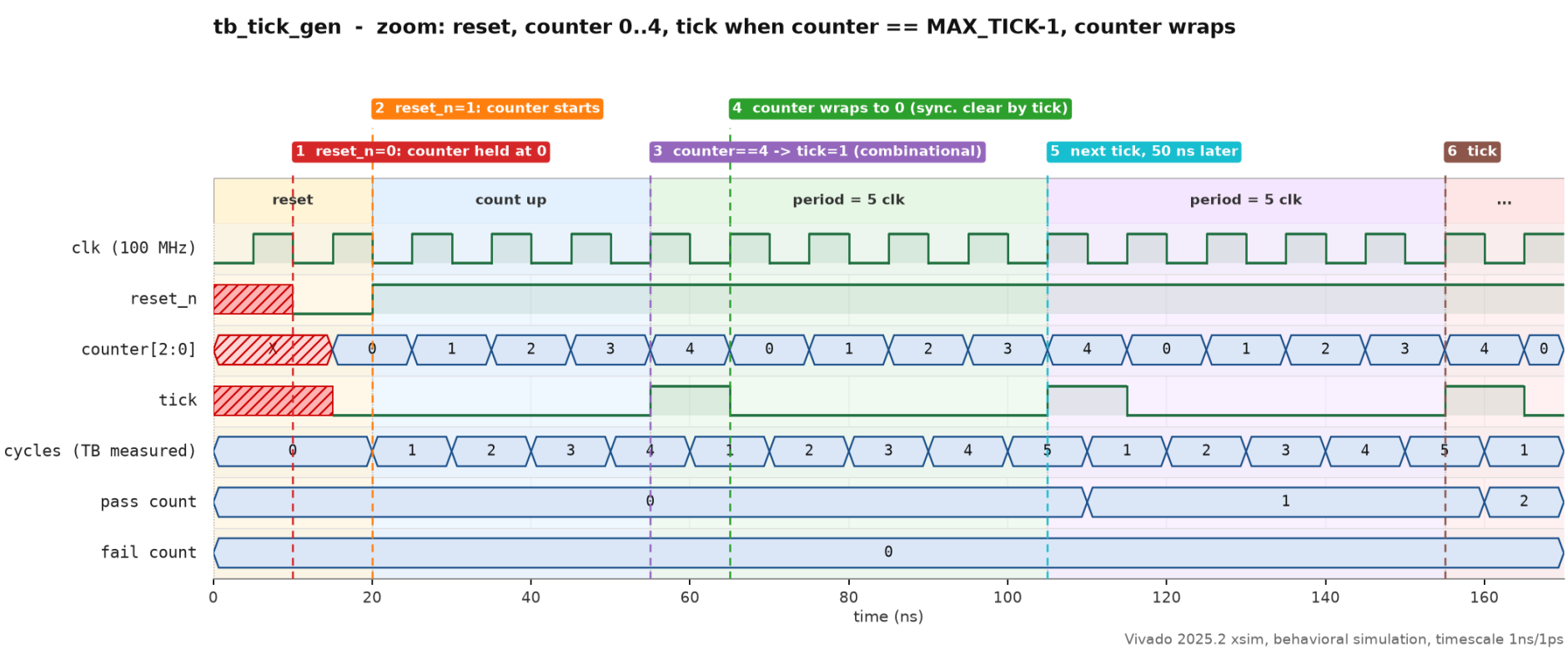


Figure 5: Zoom: the counter is held at 0 during reset (marker 1), counts 0..4 after release (2), tick is asserted combinationally while counter == 4 (3) and clears the counter on the next edge (4); the period is exactly 5 clocks (5, 6).

3.4 tb_value_latch – switch value register

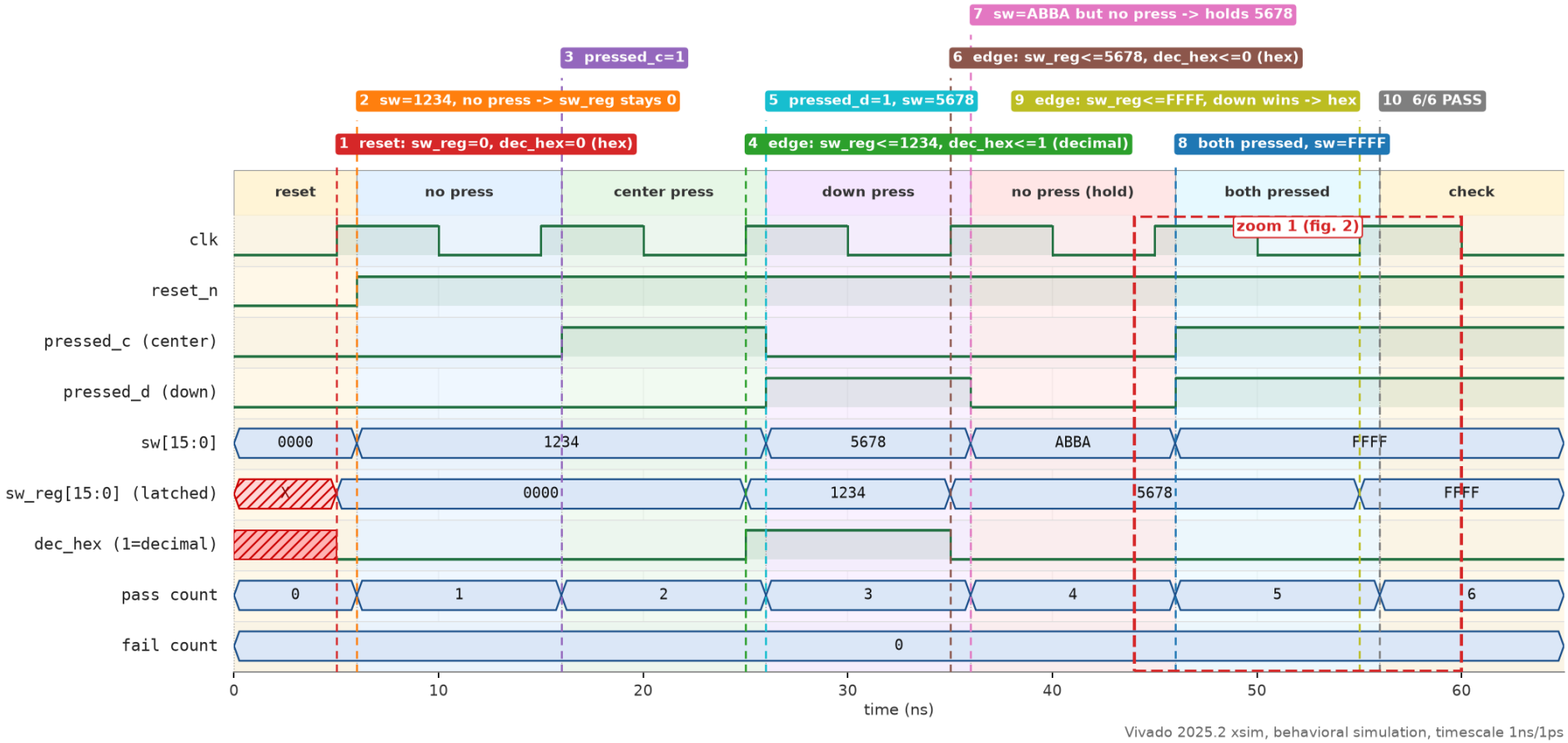


Figure 6: tb_value_latch: after reset sw_reg = 0 and hex mode (1); SW = 1234 without a press is not latched (2); pressed_c latches 1234 and selects decimal (3, 4); pressed_d latches 5678 and selects hex (5, 6); SW = ABBA without a press is ignored (7); both pulses together latch FFFF and down wins → hex (8, 9).

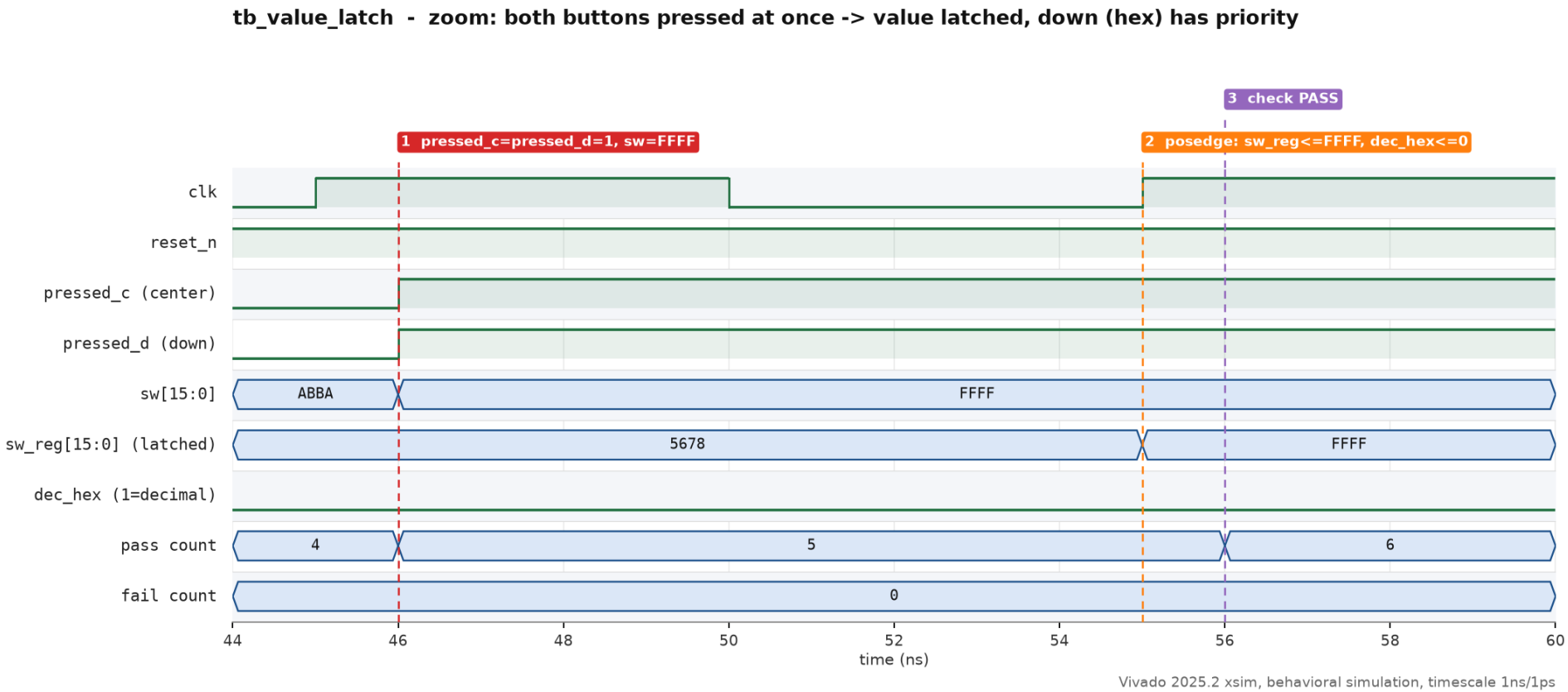


Figure 7: Zoom on the both-buttons case: at the clock edge after pressed_c = pressed_d = 1 the value is loaded and dec_hex stays 0.

3.5 tb_btn_hold_1s – 1-second hold detector

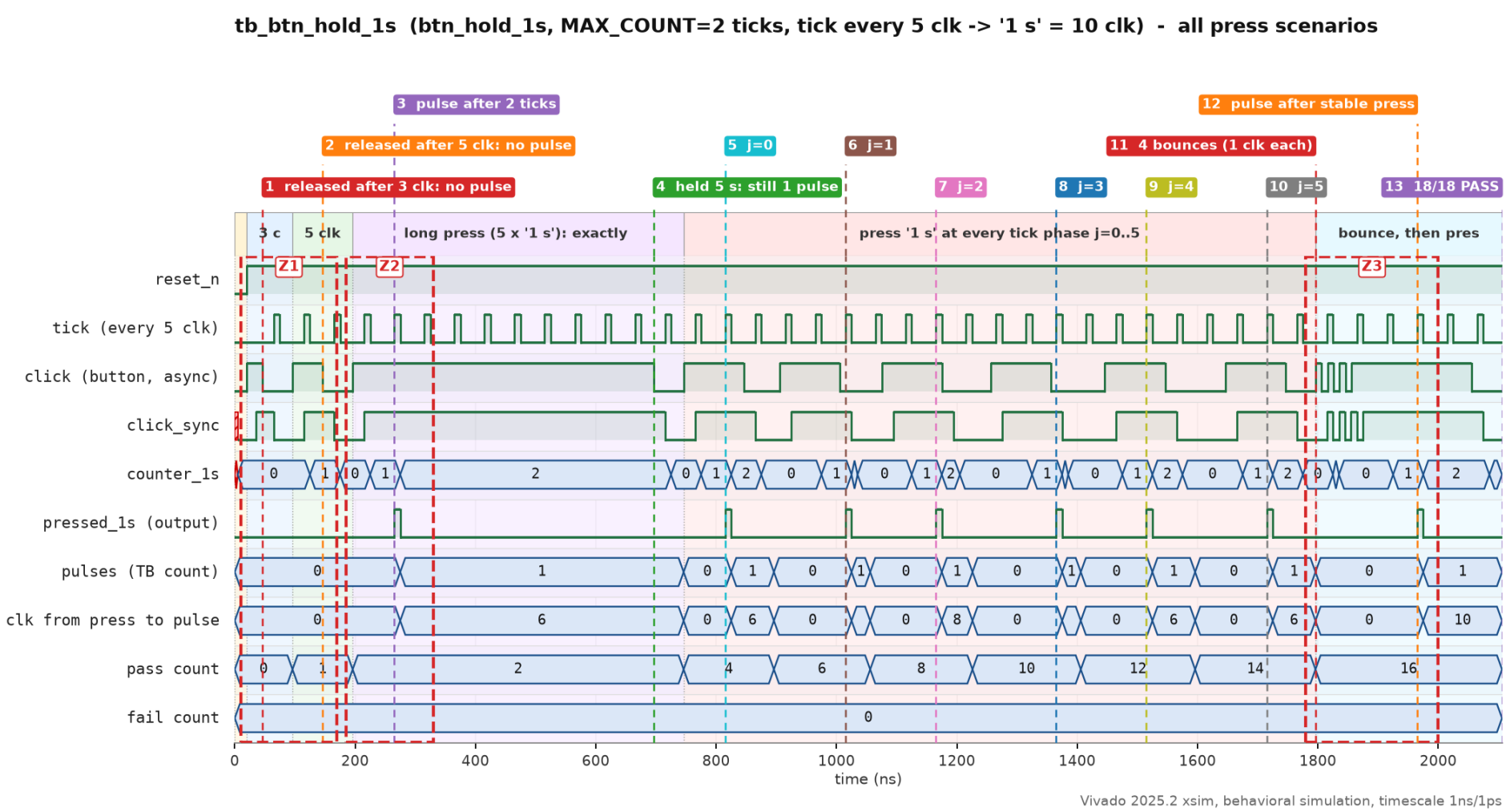


Figure 8: tb_btn_hold_1s overview (MAX_COUNT = 2 ticks, tick every 5 clocks): presses of 3 and 5 clocks give no pulse (1, 2); a long hold gives one pulse after 2 ticks and no further pulse while held (3, 4); a '1 s' press started at each of the 6 possible tick phases always gives exactly one pulse (5–10); four 1-clock bounces are ignored and the following stable press is detected (11, 12).

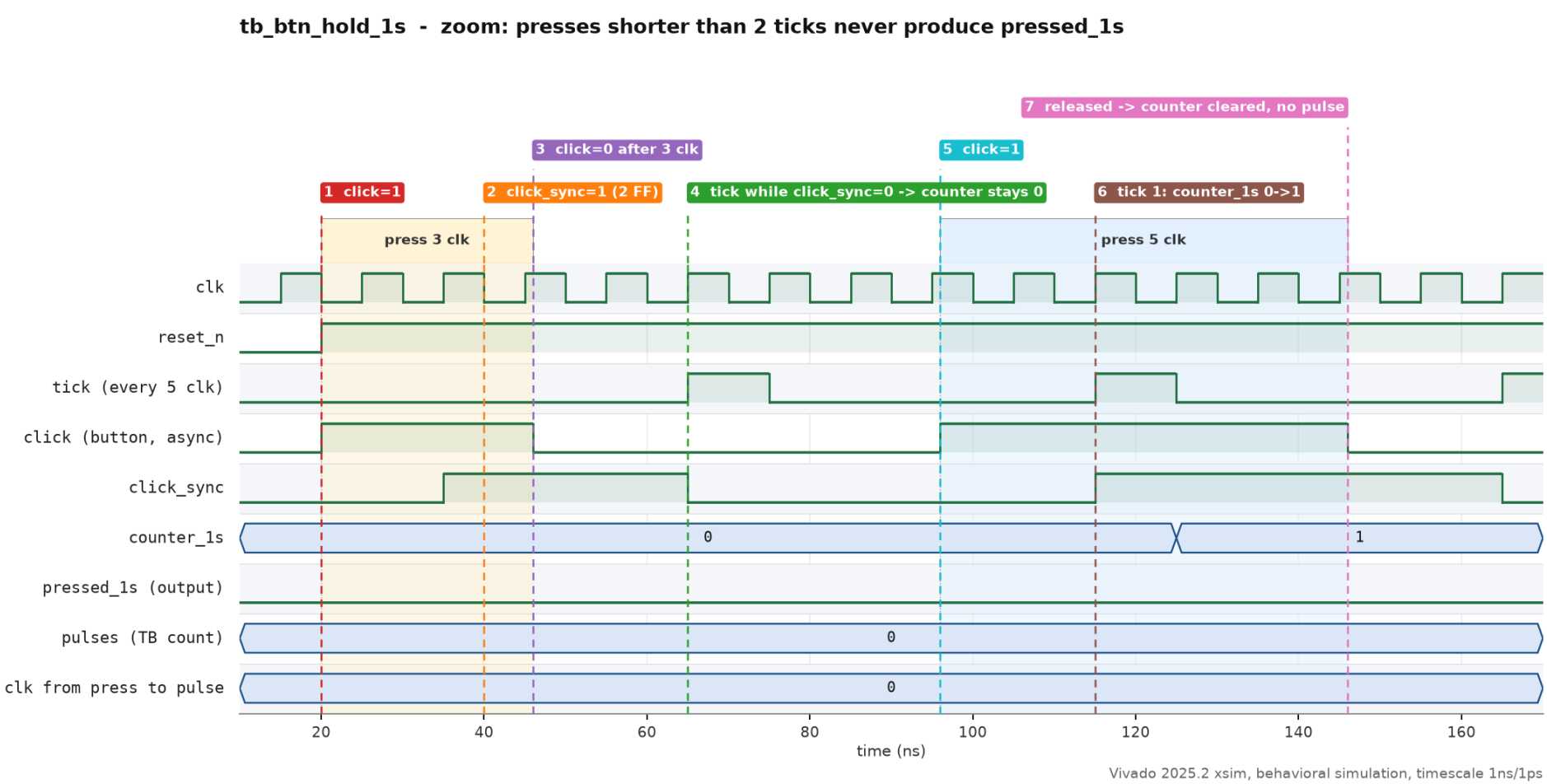


Figure 9: Zoom: the 2-FF synchronizer delays click by 2 clocks; a press released before the second tick never reaches counter_1s == 1 at a tick, so pressed_1s stays 0 and the counter is cleared on release.

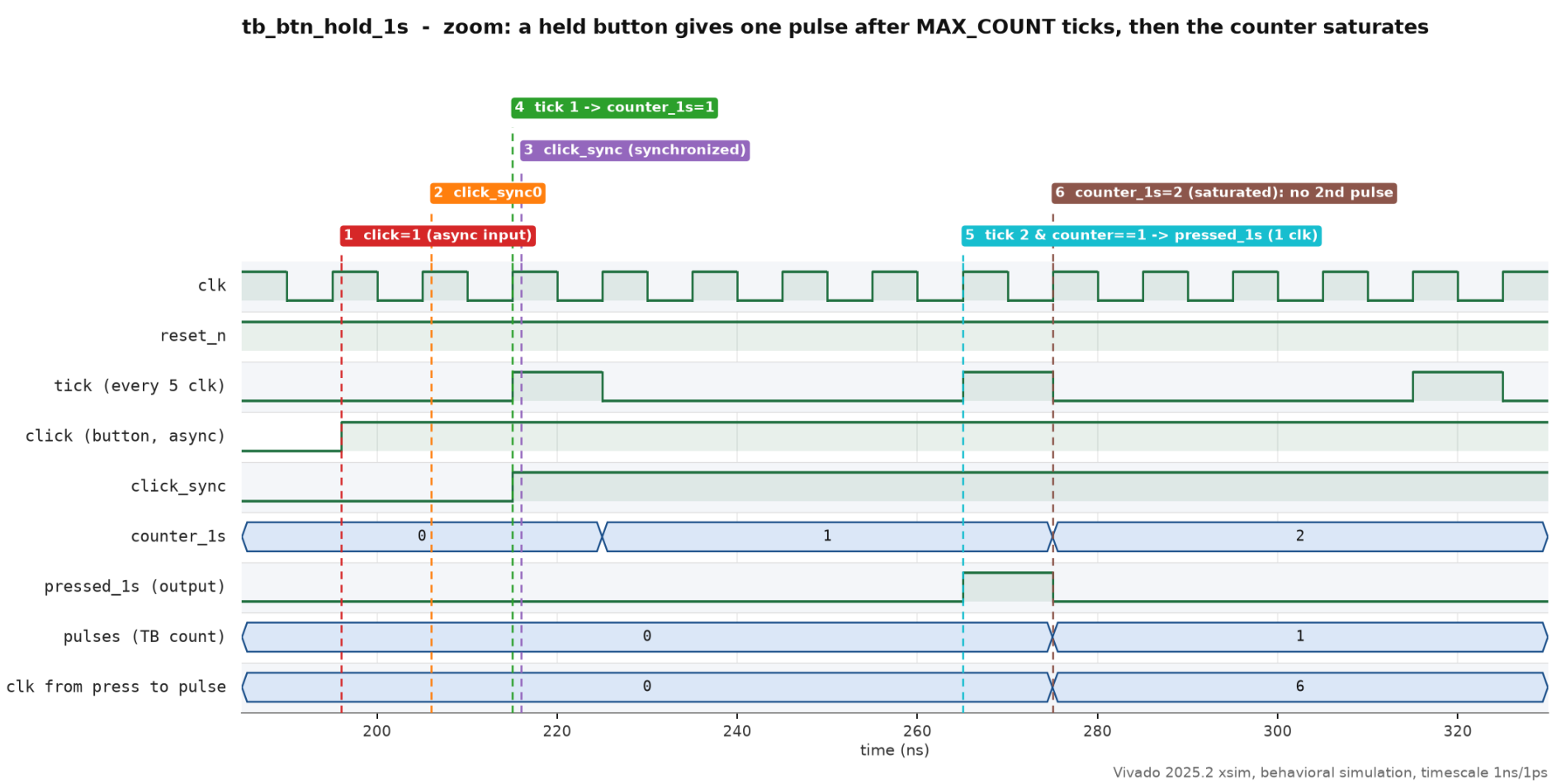


Figure 10: Zoom on a held button: click_sync0 and click_sync (markers 2, 3), tick 1 increments counter_1s (4), tick 2 with counter_1s == MAX_COUNT-1 produces the one-clock pressed_1s pulse (5), then the counter saturates at 2 so no second pulse (6).

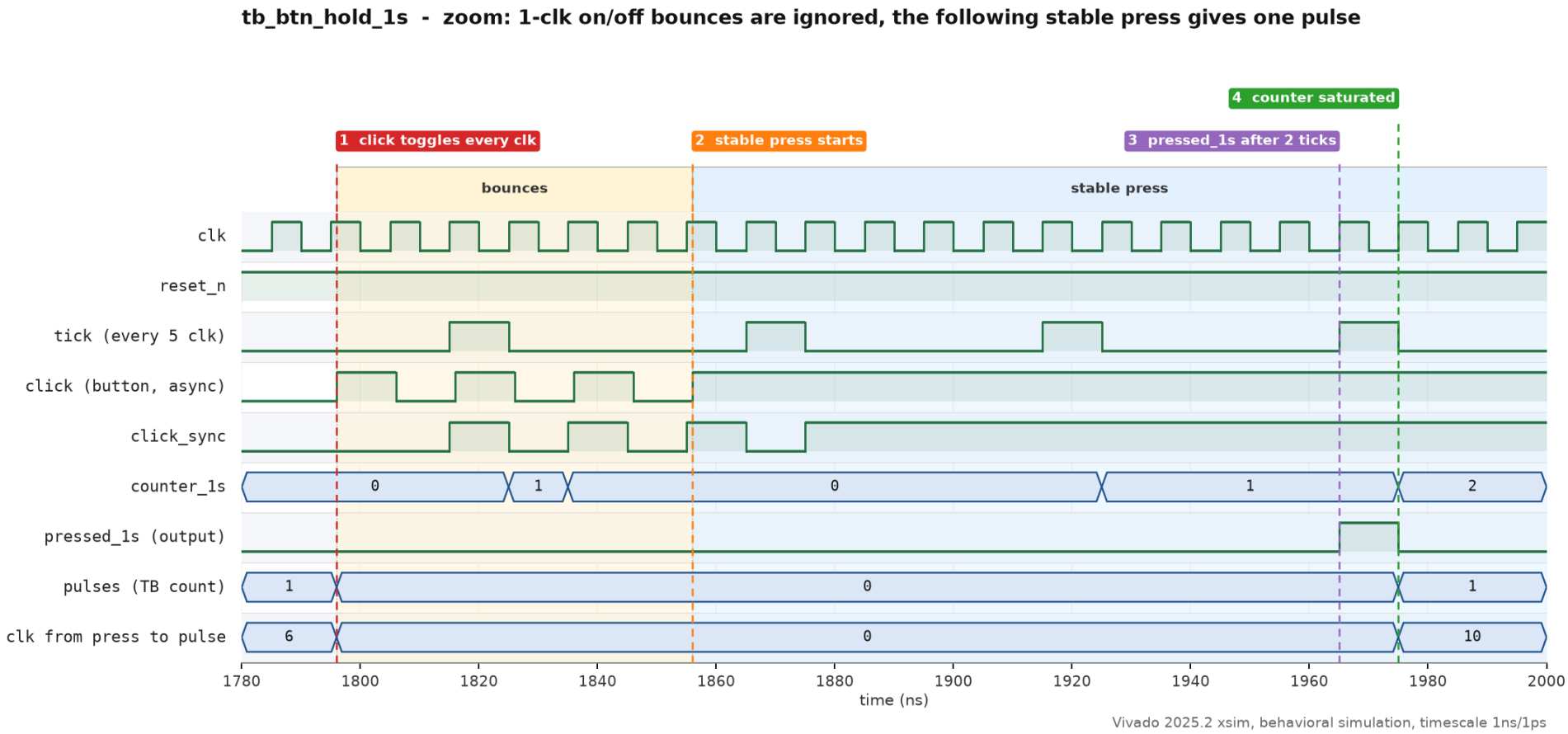


Figure 11: Zoom on contact bounce: 1-clock on/off toggles clear the counter every time; the stable press that follows is detected after two ticks.

3.6 tb_binary_2_bcd – double-dabble converter

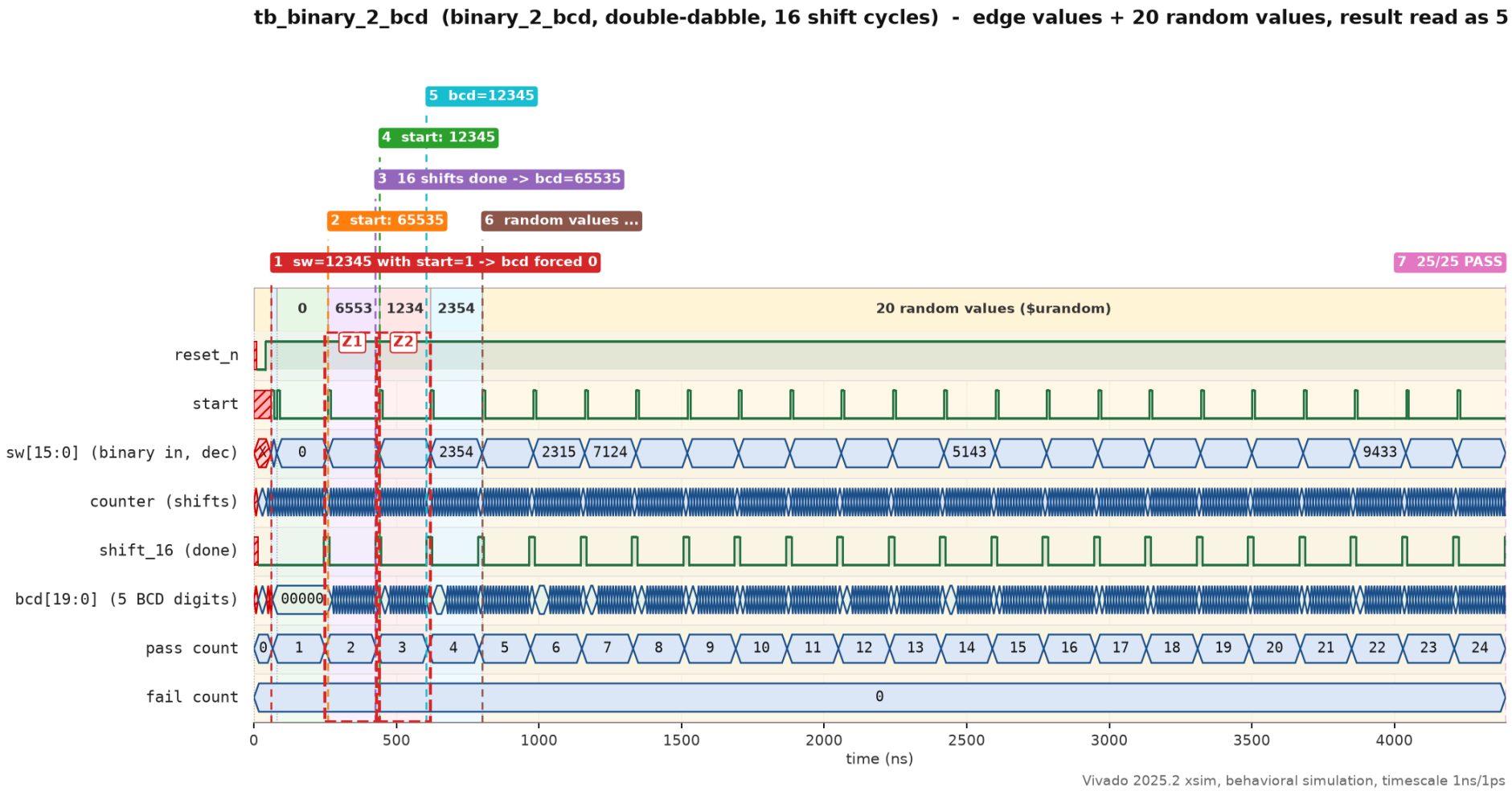


Figure 12: tb_binary_2_bcd overview: start with SW = 12345 forces bcd to 0 (1); every conversion takes 16 shift cycles after start (2–5); 20 random values follow; bcd is shown as 5 hex nibbles, which read directly as the decimal digits.

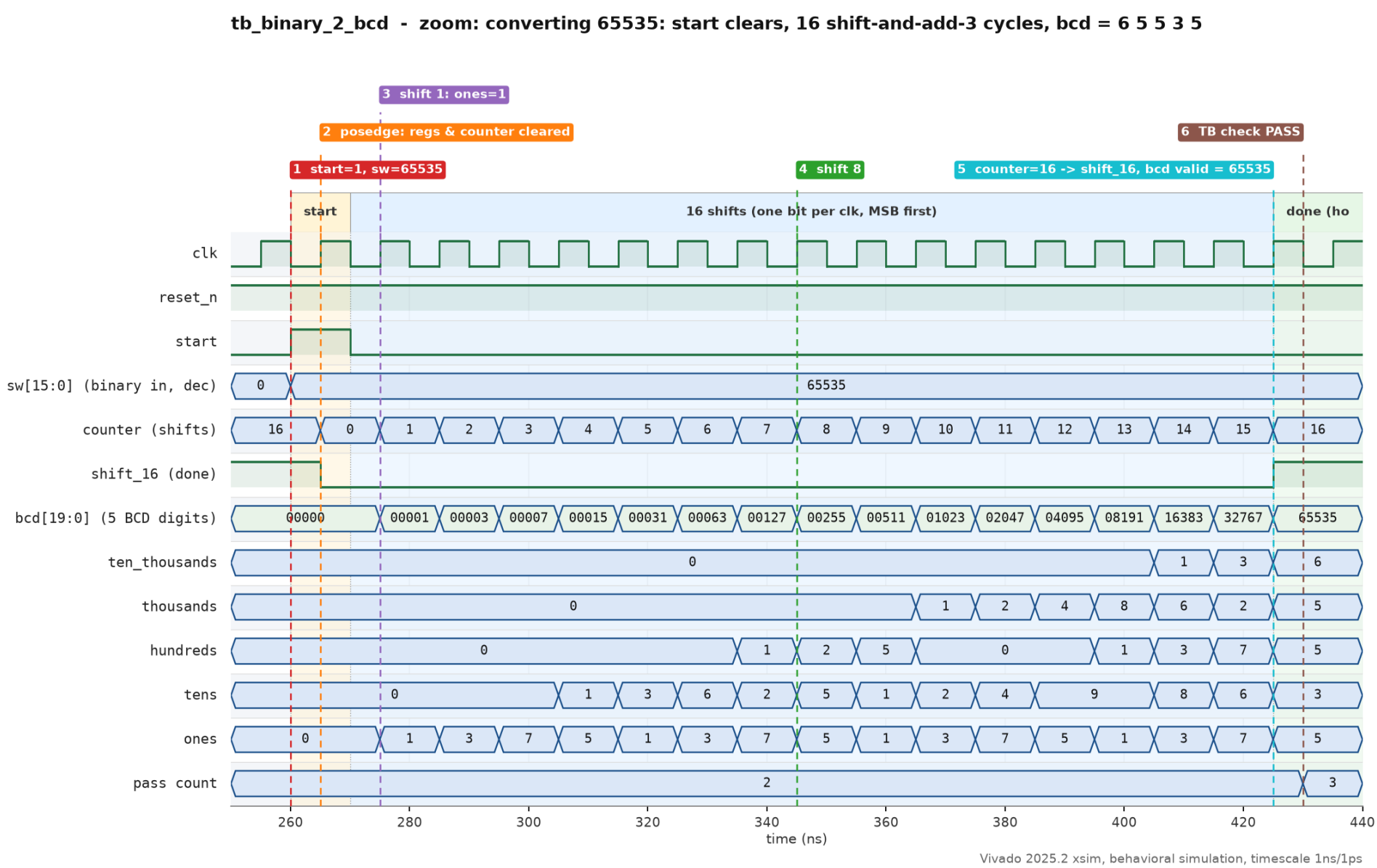


Figure 13: Zoom on 65535: start clears the digit registers and the counter (1, 2); each of the 16 clocks shifts one bit of sw in, MSB first, with add-3 applied to every digit ≥ 5; the digit registers end at 6 5 5 3 5 when counter reaches 16 and shift_16 freezes the result (5); the testbench check passes (6).

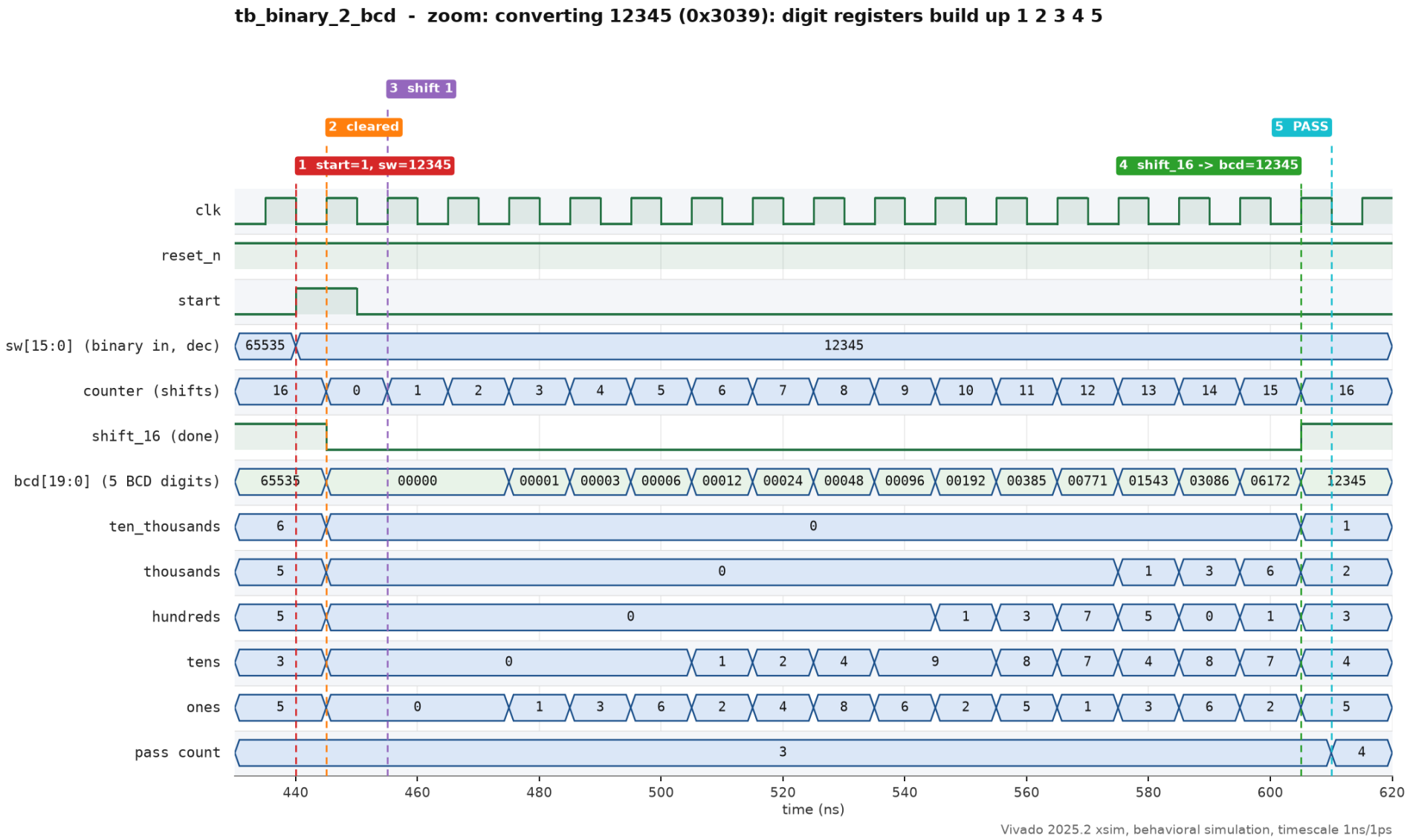


Figure 14: Zoom on 12345 (0x3039): the five digit registers build up 1 2 3 4 5 over the 16 shift cycles.

3.7 tb_display_ctrl – digit scanning

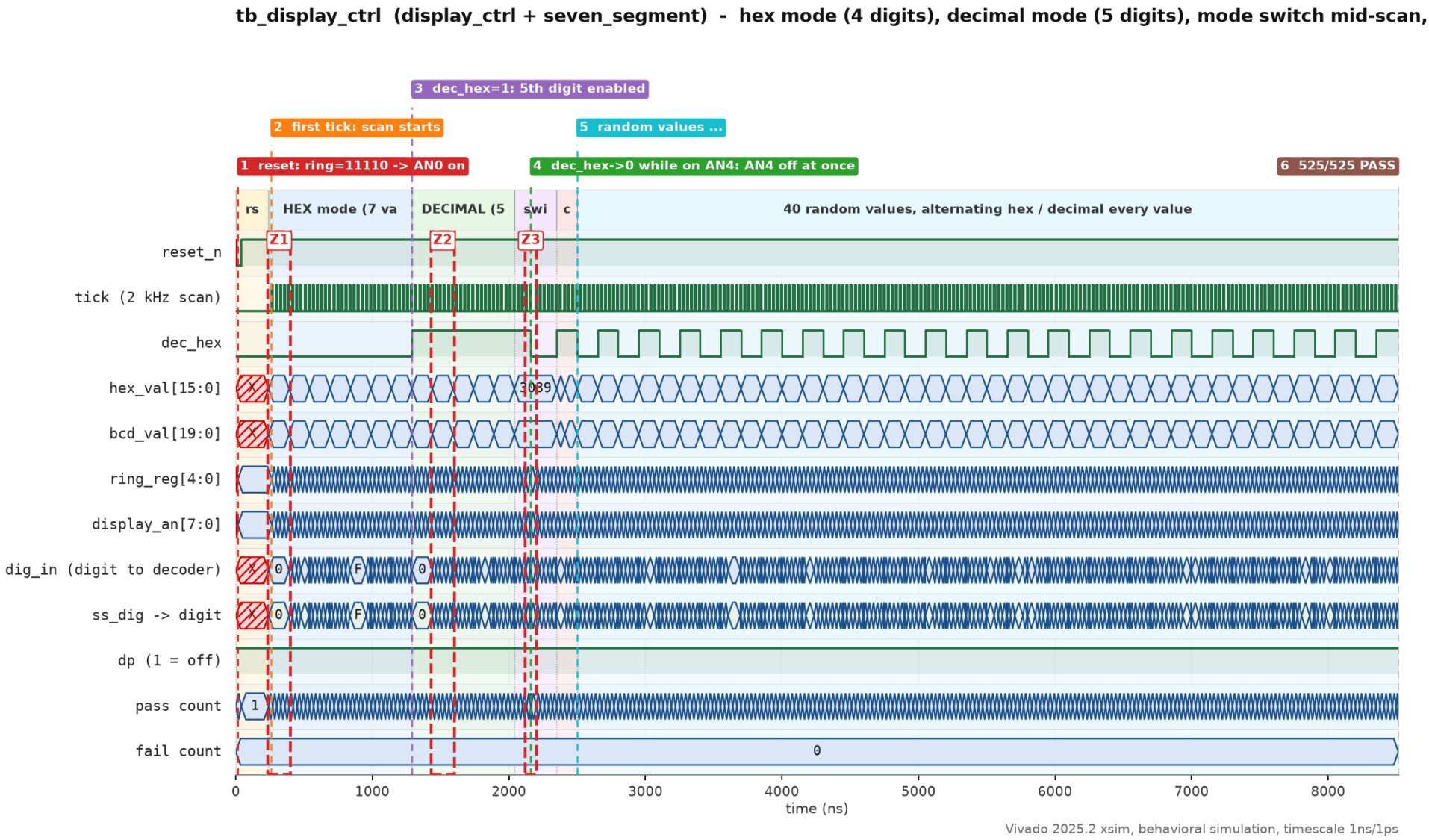


Figure 15: tb_display_ctrl overview: hex mode with 7 values (AN4 never on), decimal mode with 5 values (AN4 shows the 5th digit), a switch to hex while AN4 is active (4), a value change mid-scan, then 40 random values alternating between the two modes; 525 checks pass.

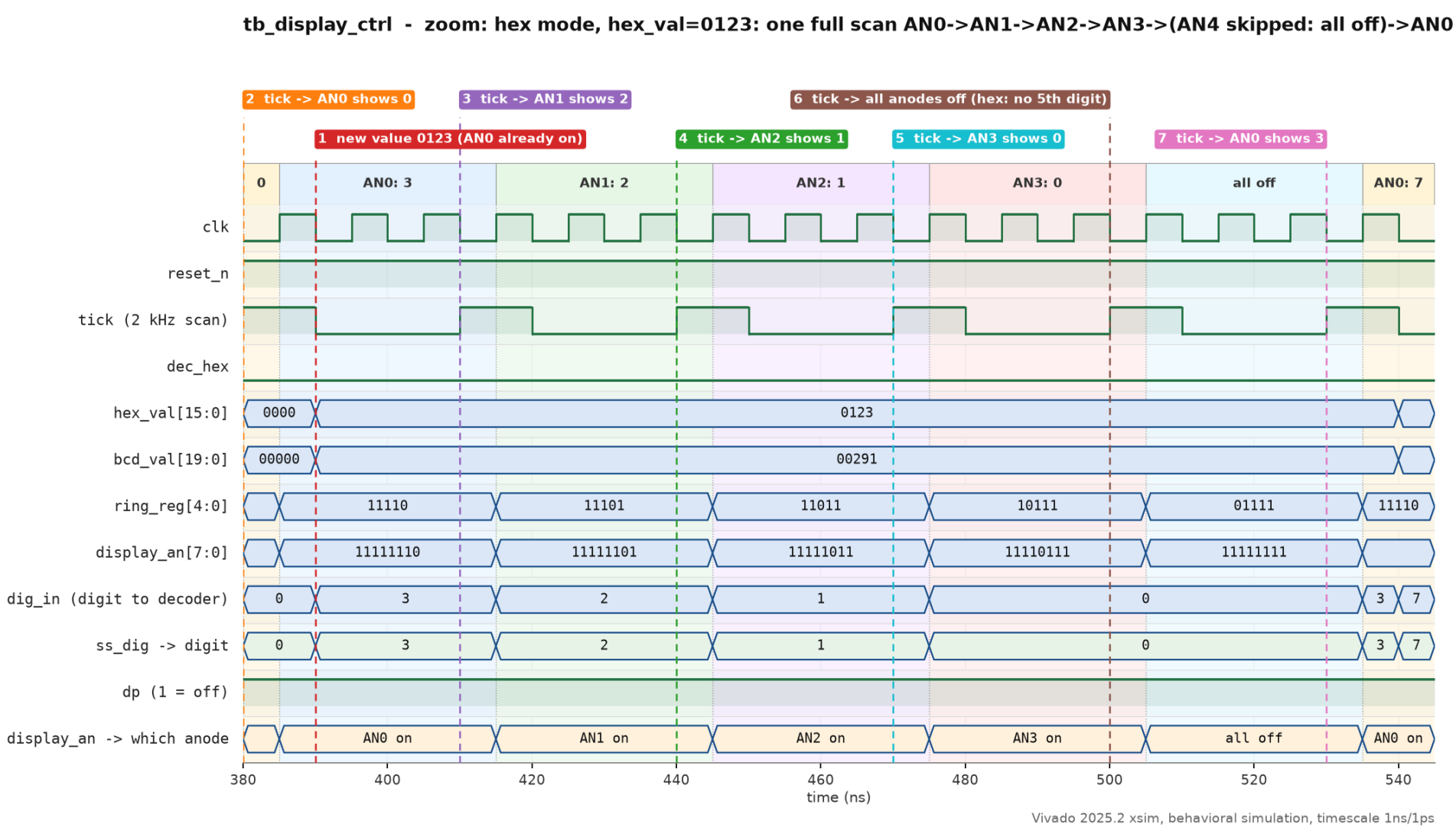


Figure 16: Zoom on one hex scan of 0123: every tick rotates ring_reg and display_an (11111110 → 11111101 → 11111011 → 11110111); in the 5th slot ring[4] = 0 but dec_hex = 0, so display_an = 11111111 (all off) and the sequence restarts at AN0 showing 3.

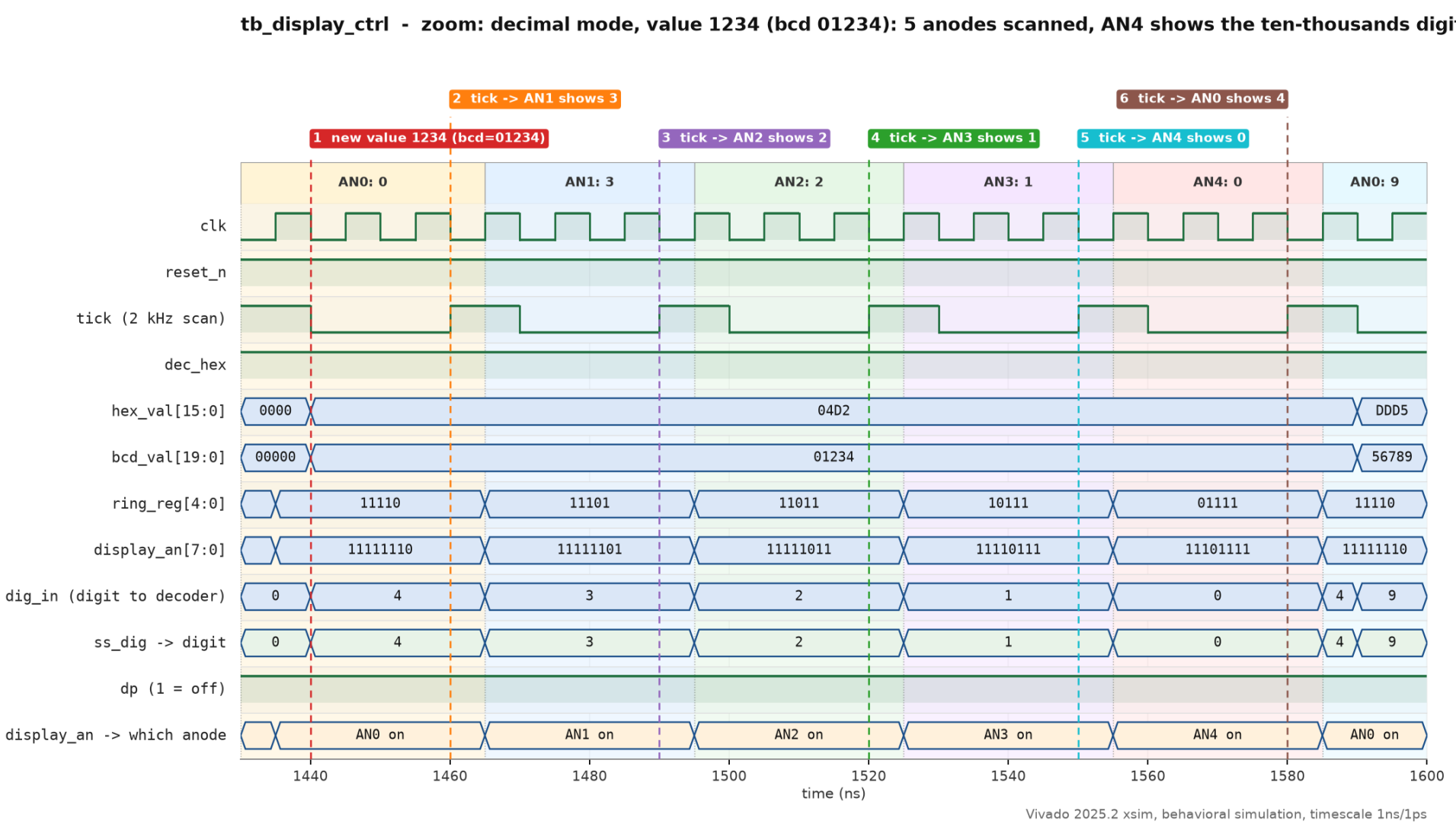


Figure 17: Zoom on one decimal scan of 1234 (bcd 01234): AN0..AN3 show 4, 3, 2, 1 and AN4 (11101111) shows the ten-thousands digit 0.

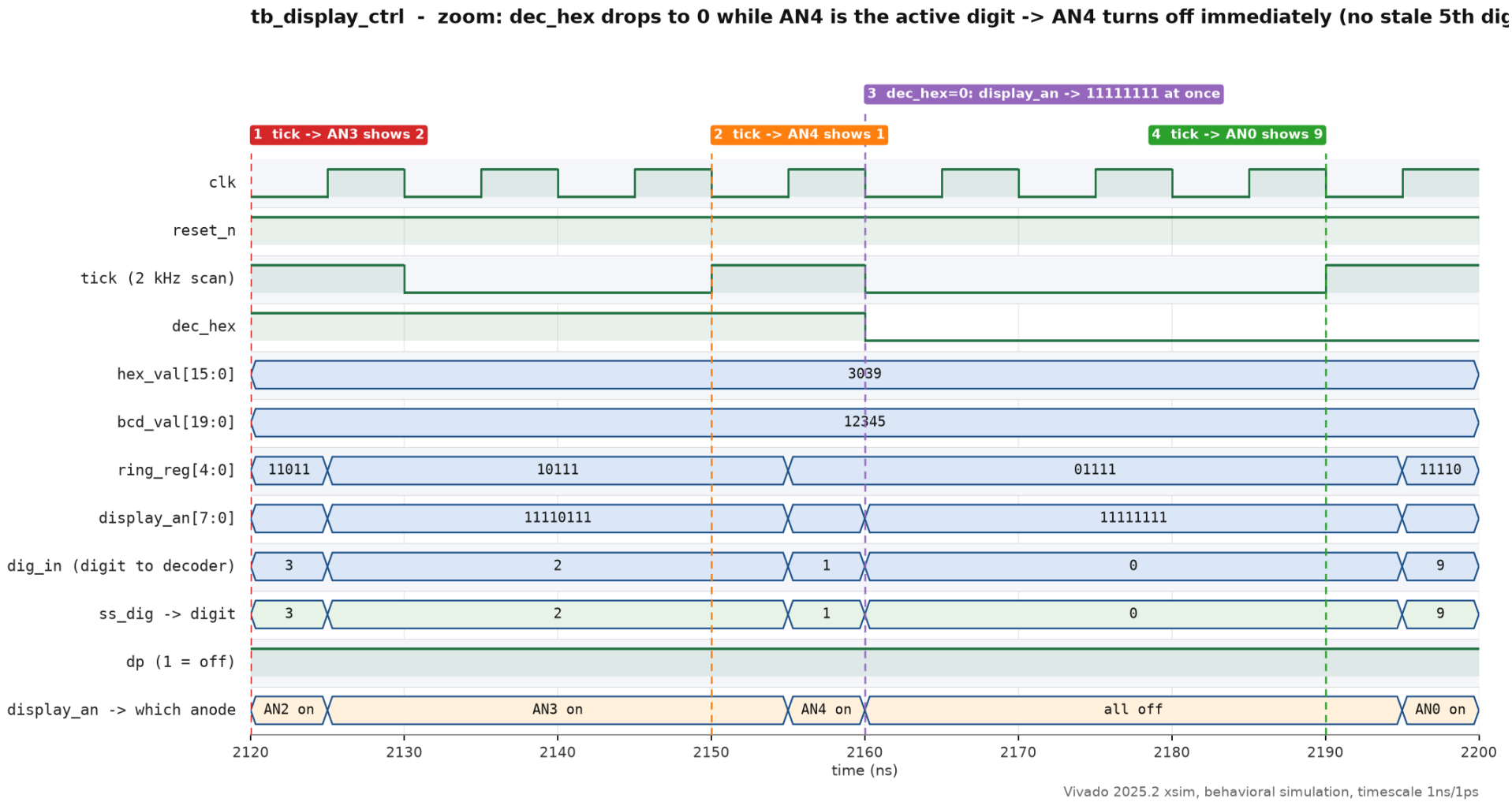


Figure 18: Zoom on the mode switch: while AN4 is the active slot, dec_hex drops to 0 and display_an becomes 11111111 in the same cycle – no stale 5th digit is ever shown in hex mode.

3.8 tb_chip_top – full system

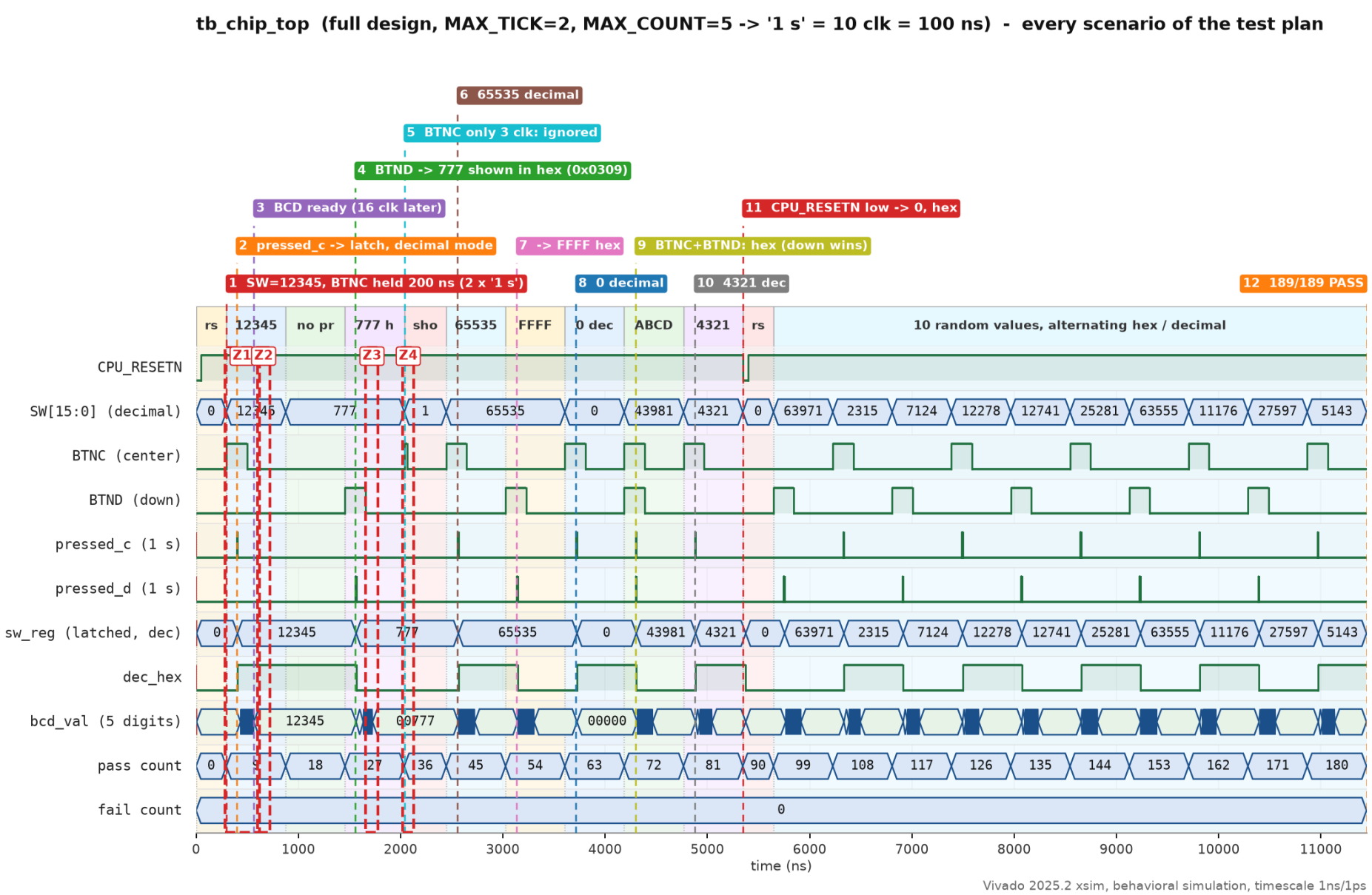


Figure 19: tb_chip_top overview (MAX_TICK = 2, MAX_COUNT = 5 → '1 s' = 100 ns): each shaded section is one scenario of the test plan. The testbench records, for every anode, which digit was displayed and compares with the expected value (189 checks).

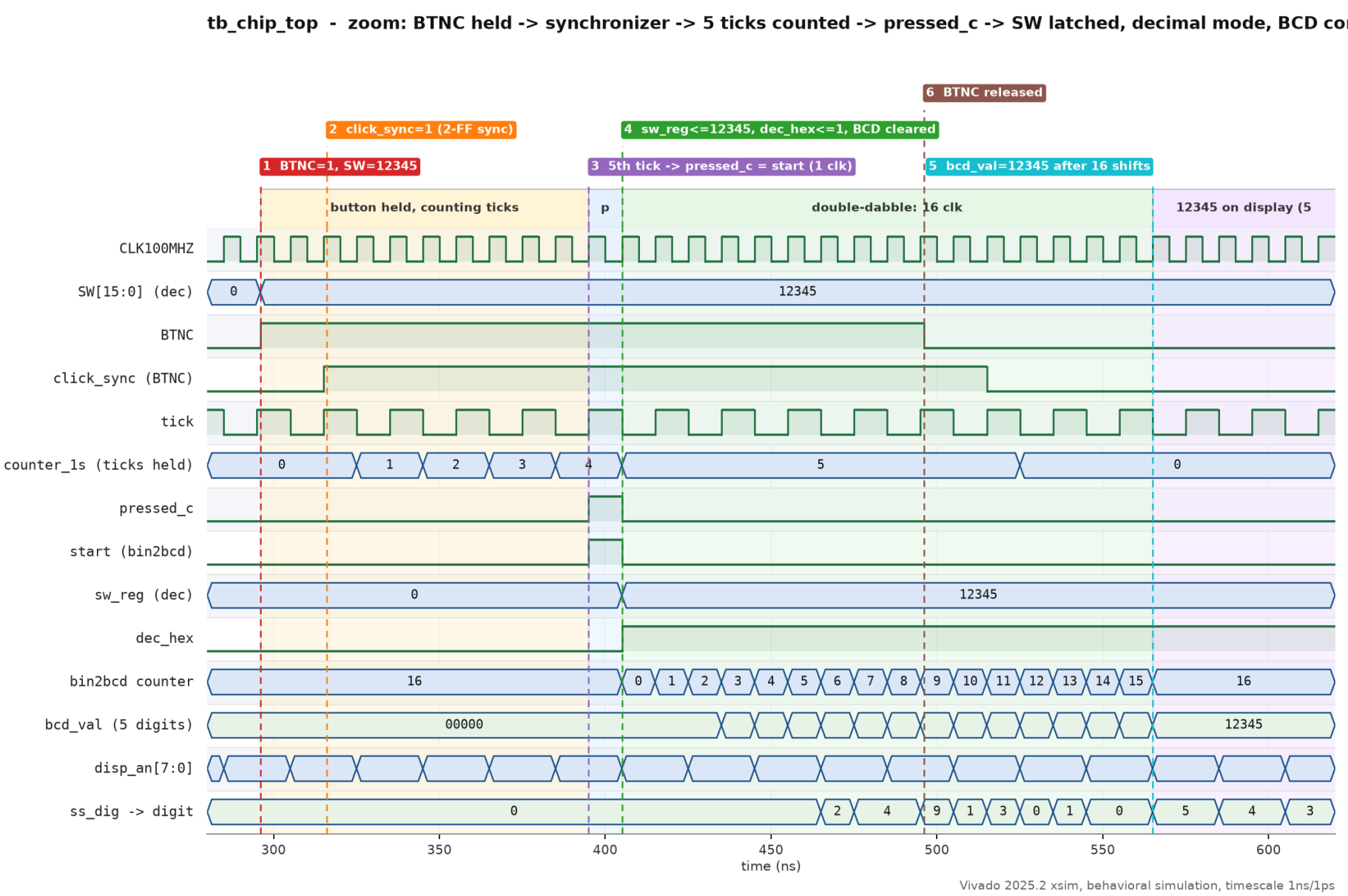


Figure 20: Zoom on a center press: BTNC rises (1), is synchronized (2), five ticks are counted and pressed_c = start pulses for one clock (3); on the next edge sw_reg = 12345, dec_hex = 1 and the converter is cleared (4); 16 clocks later bcd_val = 12345 (5) and the display scans 5 digits while the button is still held (6).

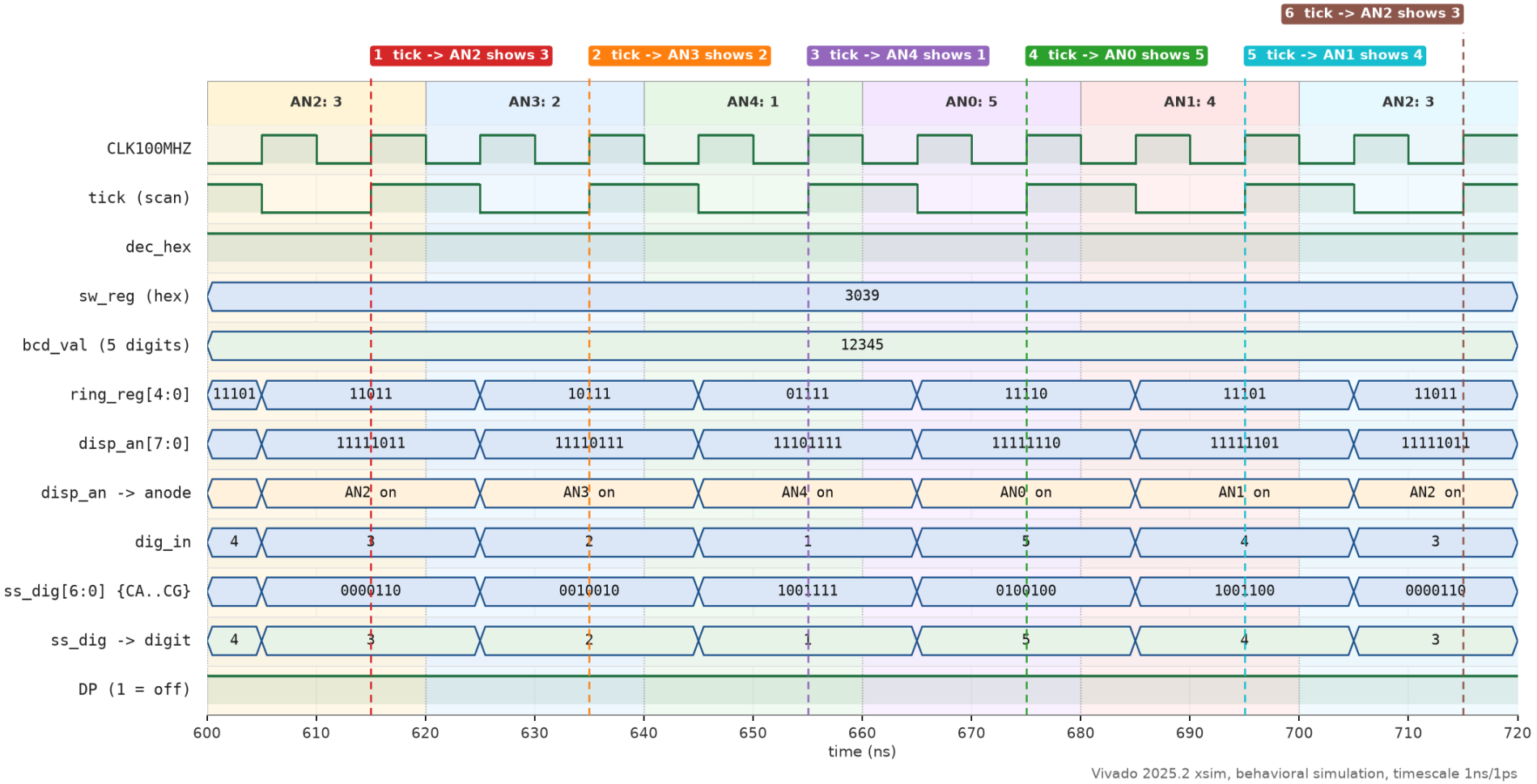


Figure 21: Zoom on the decimal scan of 12345: one anode per tick, AN0..AN4 show 5 4 3 2 1, the cathode pattern decodes to the same digit, DP stays off.

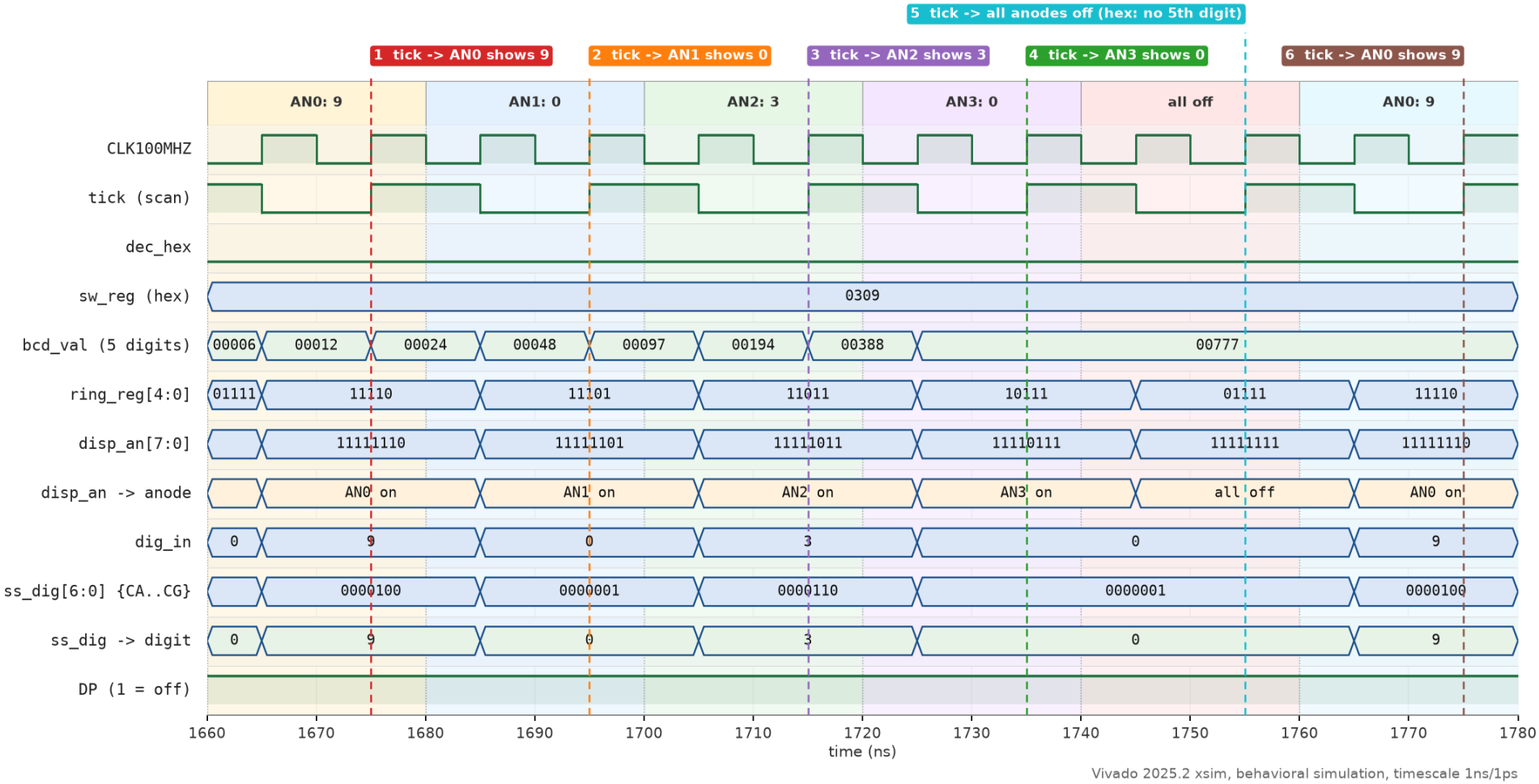


Figure 22: Zoom on the hex scan of 777 = 0x0309 after a down press: AN0..AN3 show 9 0 3 0 and the 5th slot has all anodes off.

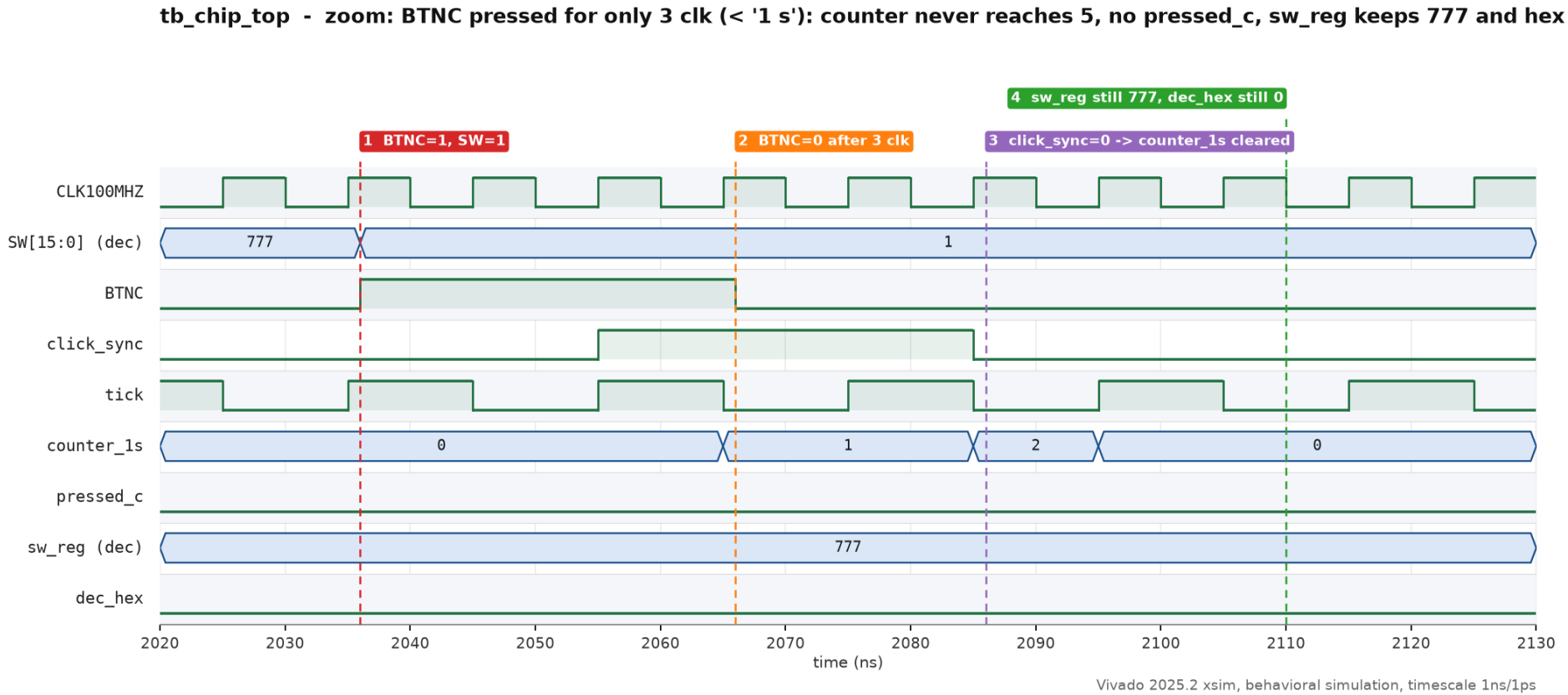


Figure 23: Zoom on a 3-clock BTNC press: the counter never reaches 5, pressed_c stays 0, sw_reg keeps 777 and the mode stays hex.

4. RTL analysis – lint and elaborated design

4.1 Lint

Vivado's RTL linter (RTL ANALYSIS → Run Linter, synth_design -lint) was run on chip_top with all sub-modules: 0 rules violated, 0 waived (2_Lint_and_RTL_Schematics/vivado_lint_report_chip_top_with_all_submodules.txt). As an additional check every module was also linted with Verilator -Wall: the six sub-modules are completely clean (0 errors, 0 warnings). chip_top gets two PROCASSINIT style notes on the reset-synchronizer flip-flops, which have a power-up initial value and are also written in an always block – this is intentional (FPGA registers have a defined power-up state and the synchronizer deliberately has no reset of its own).

Linter	Scope	Errors	Warnings	Report
Vivado RTL Linter	chip_top + all sub-modules	0	0 violations	vivado_lint_report_chip_top_with_all_submodules.txt
Verilator -Wall	each of the 7 modules	0	0 on the 6 sub-modules; 2 style notes (PROCASSINIT) on chip_top	verilator_lint/verilator_lint_<module>.log

4.2 Elaborated design checks

Open Elaborated Design on chip_top (169 instances) followed by Report Methodology and Report DRC: both report 0 checks found. Report Noise (SSN) with the pin constraints applied: all 36 pins PASS with > 80 % margin. The RTL schematics below are the elaborated schematics of every module (RTL ANALYSIS → Schematic).

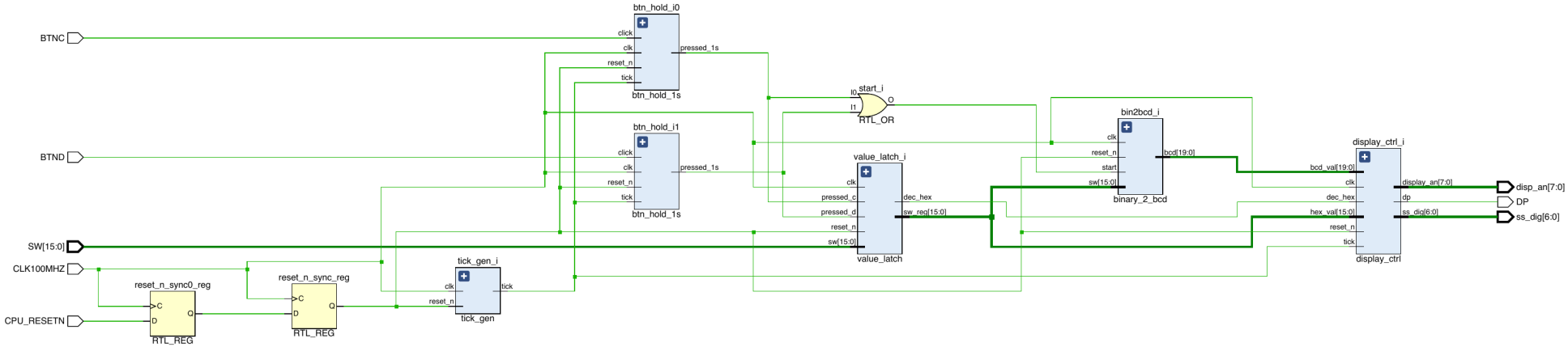


Figure 24: elaborated chip_top: reset synchronizer, tick_gen, two btn_hold_1s, value_latch, binary_2_bcd and display_ctrl; start = pressed_c | pressed_d.

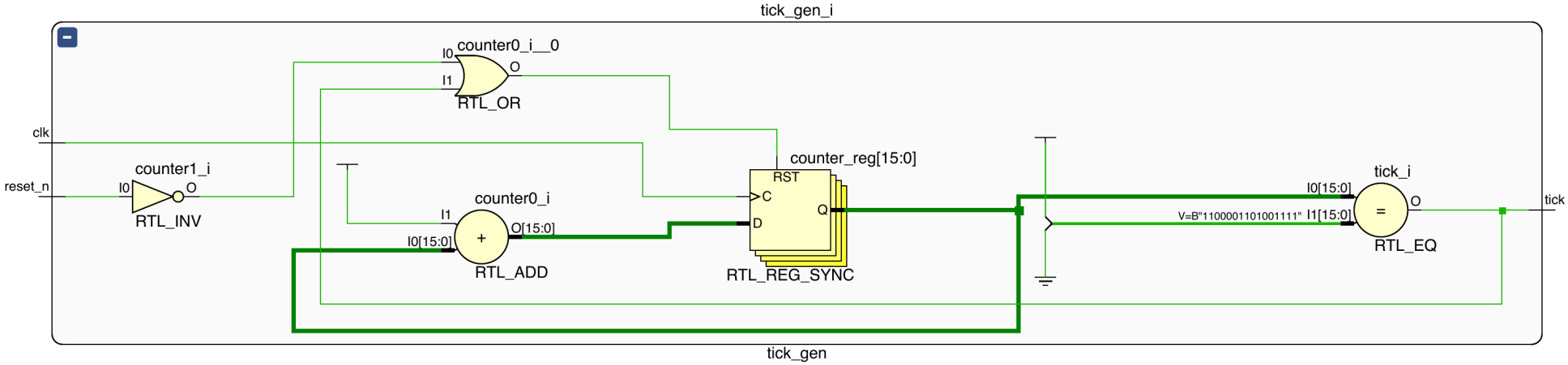


Figure 25: Elaborated tick_gen: 16-bit counter, equality compare and synchronous clear.

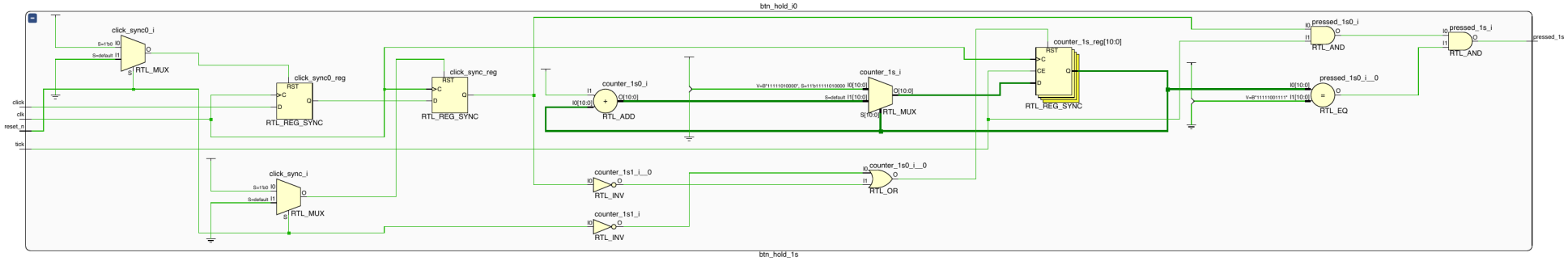


Figure 26: Elaborated btn_hold_1s: 2-FF synchronizer, 11-bit tick counter with saturation, compare and output AND.

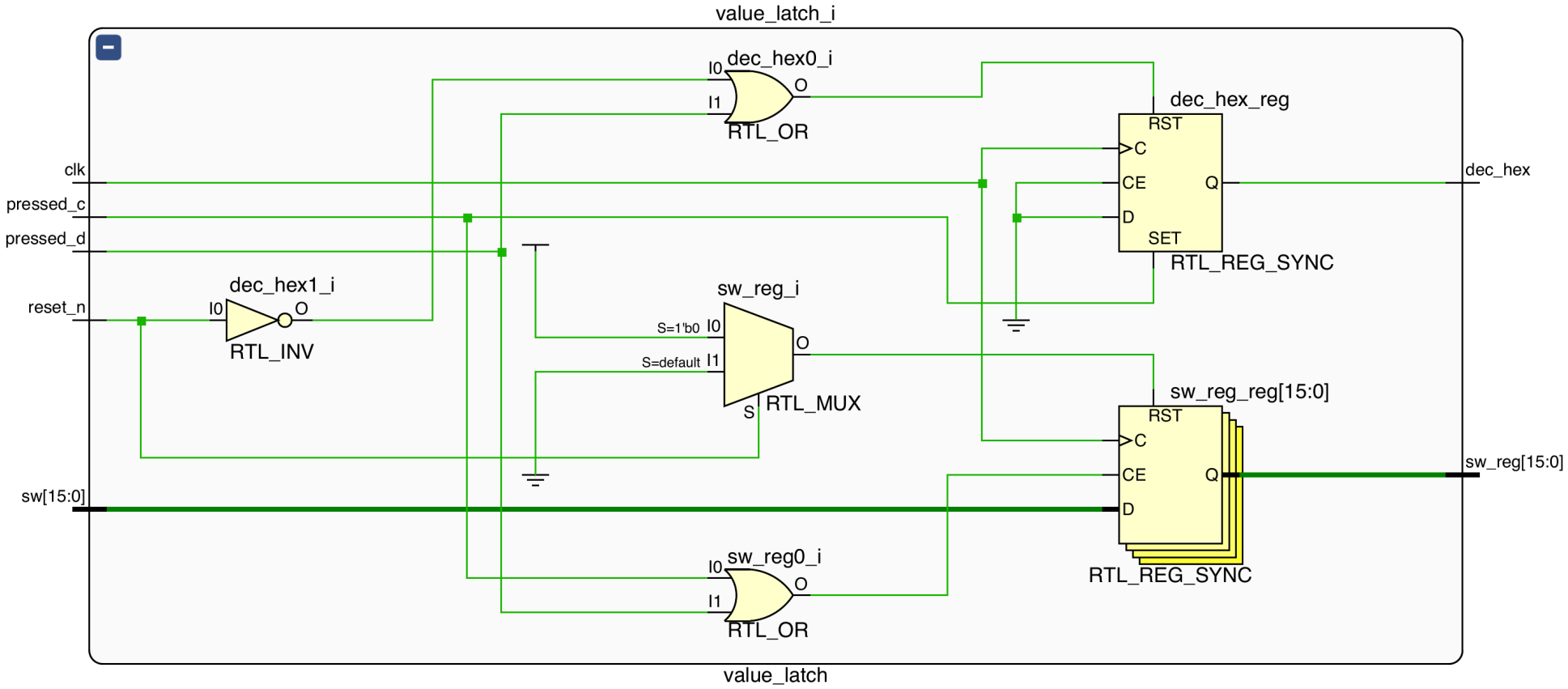


Figure 27: Elaborated value_latch: 16-bit register with clock enable and the dec_hex mode flip-flop.

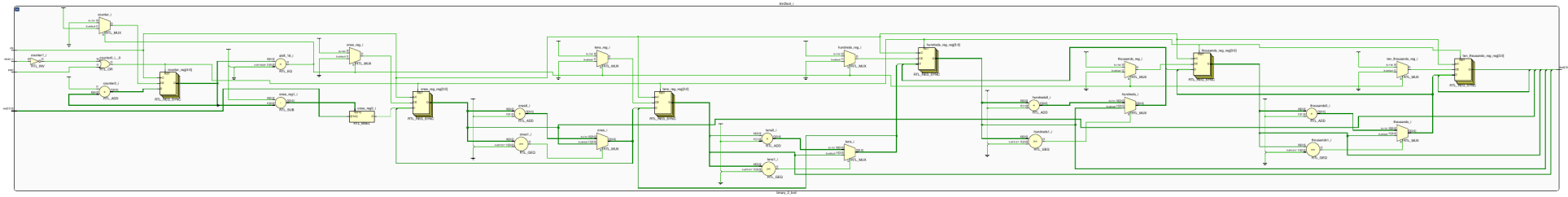


Figure 28: Elaborated binary_2_bcd: cycle counter, bit selector, four add-3 stages and the five digit registers.

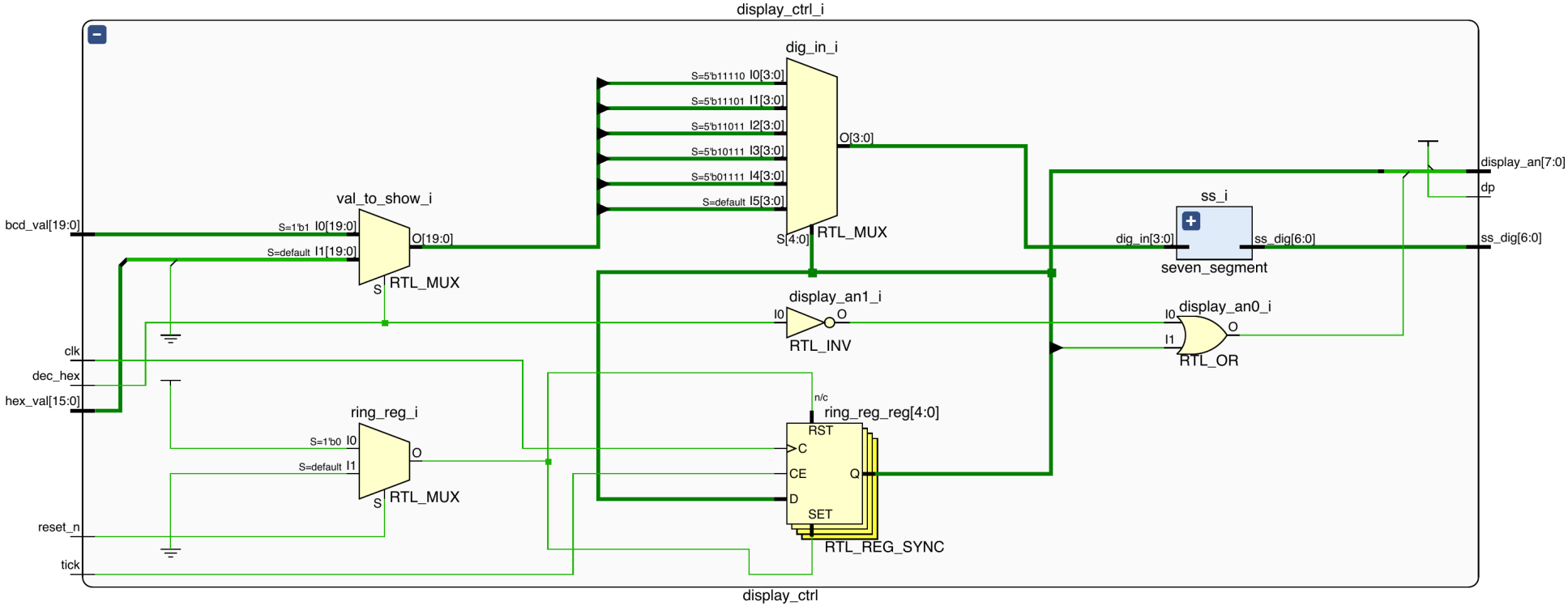


Figure 29: Elaborated display_ctrl: ring register, value mux, nibble select, anode logic and the seven_segment instance.

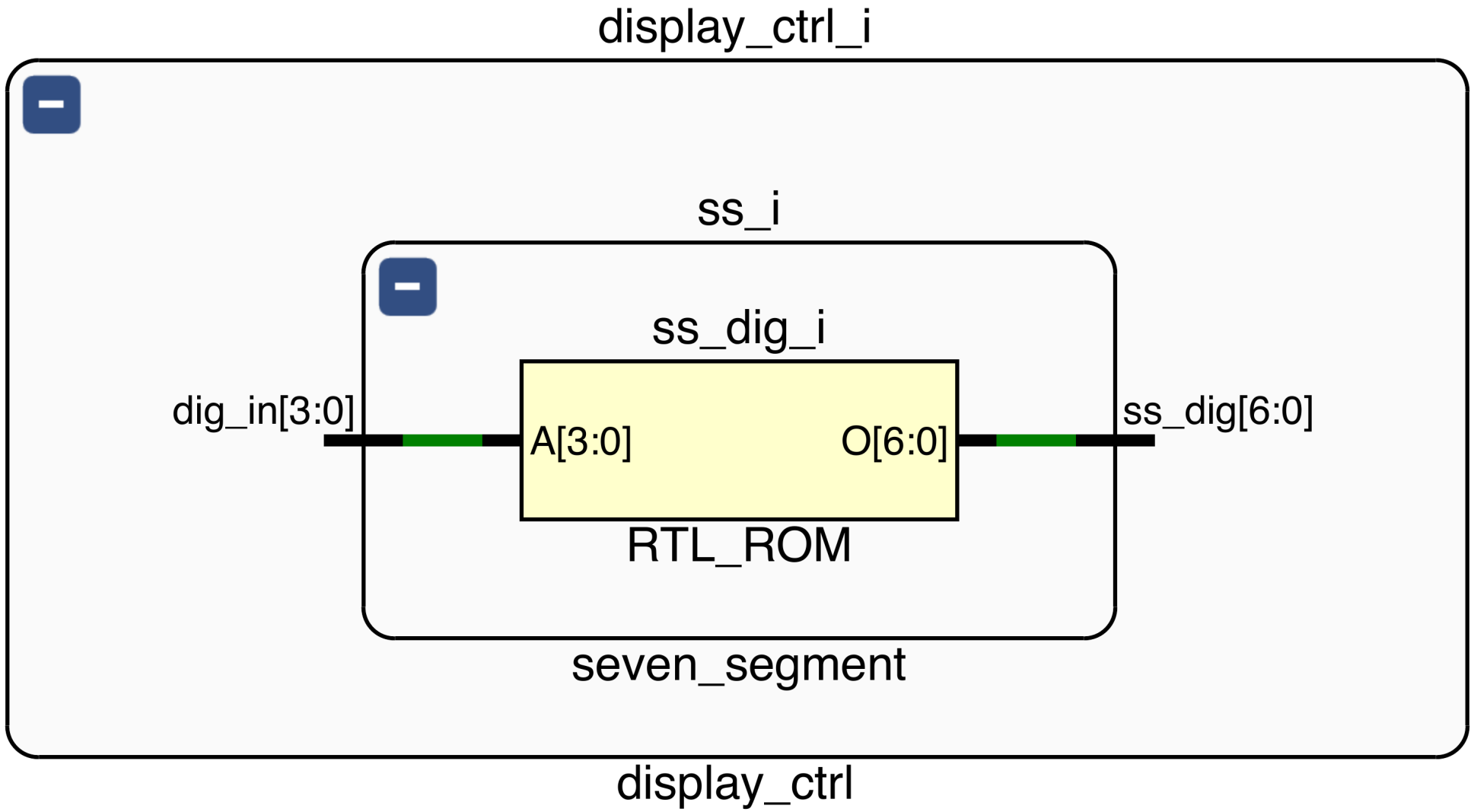


Figure 30: Elaborated seven_segment: the case statement as a ROM-style decoder.

5. Synthesis

Run Synthesis (synth_chip_top, Vivado Synthesis Defaults) finished with 0 errors, 0 critical warnings and 4 warnings. The 4 warnings are all [Synth 8-3917] 'port driven by constant 1' for DP and disp_an[7], disp_an[6], disp_an[5]: these outputs are intentionally tied off (decimal point and the three unused digits are always off), so the report is clean. Opening the synthesized design and running the report set gives:

Report	Result
Utilization (synth, utilization_synth.rpt)	83 slice LUTs (110 LUT primitives before LUT combining), 91 flip-flops (87 FDRE + 4 FDSE), 4 CARRY4, 1 MUXF7, 36 I/O, 1 BUFG, 0 latches, 0 BRAM/DSP
Timing summary (synth estimate)	sys_clk_pin 10 ns: WNS +5.262 ns, WHS +0.144 ns – met
DRC	0 checks
Methodology	34 × TIMING-18 'missing input or output delay' – see note below
Clock networks / interaction	one clock, CLK100MHZ → BUFG, no clock interaction issues
Noise (SSN)	all pins PASS
Power (synth estimate)	0.110 W

TIMING-18 note: the 34 methodology warnings list every I/O except the clock (20 inputs: SW, BTNC, BTND, CPU_RESETN; 16 outputs: anodes and cathodes). They are asynchronous mechanical switches/buttons and LED drivers with no timing relationship to the clock, so set_input_delay / set_output_delay constraints are not meaningful for them; the inputs are synchronized inside the design (buttons, reset) or only sampled when a button pulse occurs (switches). The warning is informational and the same 34 checks appear after implementation.

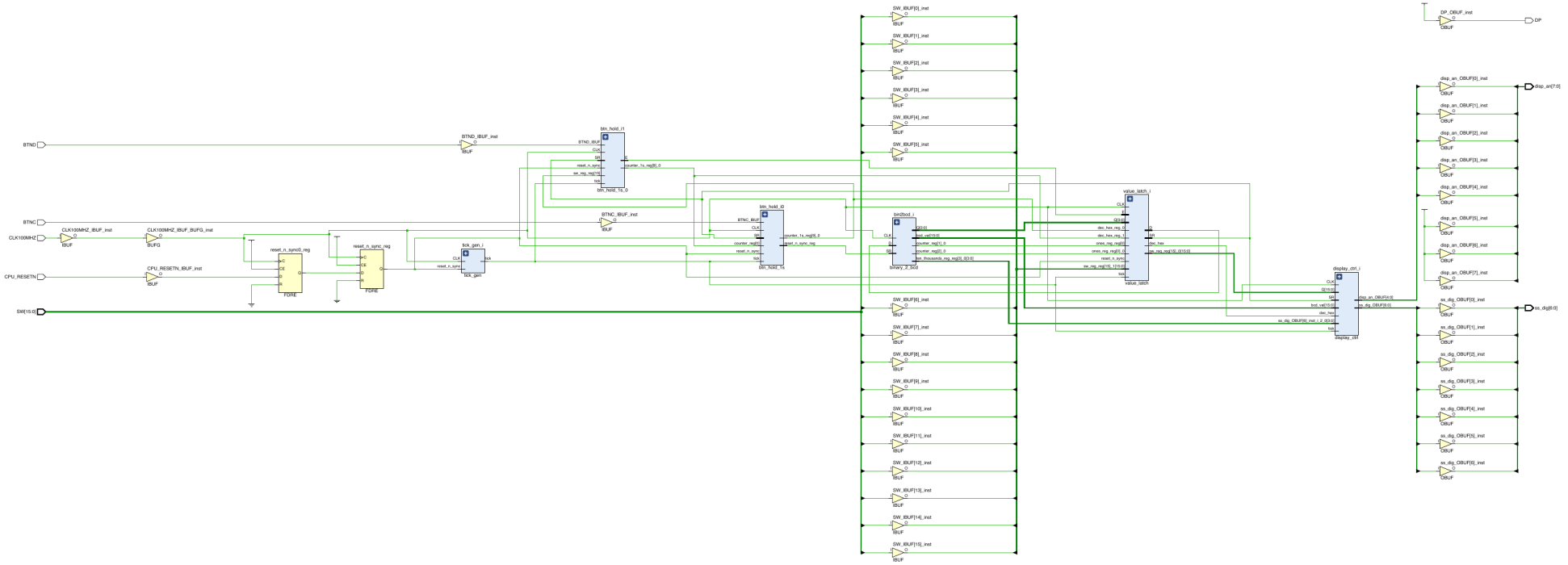


Figure 31: Synthesized chip_top: IBUF/OBUF on every pin, the clock BUFG, the two reset flip-flops and the six sub-module instances. Per-module synthesized schematics are in 3_Synthesis_Reports.

6. Implementation

Run Implementation (impl_chip_top: opt_design, place_design, phys_opt_design, route_design) finished with 0 errors, 0 critical warnings and 0 warnings. Opening the implemented design and running the full report set:

Report	Result
Timing summary (routed)	WNS +5.326 ns, TNS 0, 0/237 failing; WHS +0.163 ns, THS 0; WPWS +4.500 ns – all constraints met
Critical path	tick_gen counter_reg[3] → tick → value_latch sw_reg[11] CE: 4.397 ns, 4 logic levels (78 % routing); the design could run at ≈ 214 MHz
Route status	288 logical nets, 184 routable nets, 184 fully routed, 0 routing errors
DRC (routed)	0 checks
Methodology (routed)	34 × TIMING-18 (same I/O-delay note as after synthesis)
Utilization (placed, utilization_placed.rpt)	80 slice LUTs (110 LUT primitives, 30 dual-output pairs), 95 FFs (88 FDRE + 7 FDSE), 35 slices, 4 CARRY4, 1 MUXF7, 36 IOB, 1 BUFG
Control sets	9 unique control sets
Clock utilization	1 BUFG (CLK100MHZ), no MMCM/PLL
Power (routed)	0.109 W total: 0.097 W static, 0.011 W dynamic (0.010 W of it in the I/O); junction 25.5 °C
I/O	all 36 ports placed on the XDC pins, LVCMOS33 (SW8/SW9 LVCMOS18), no unconstrained ports

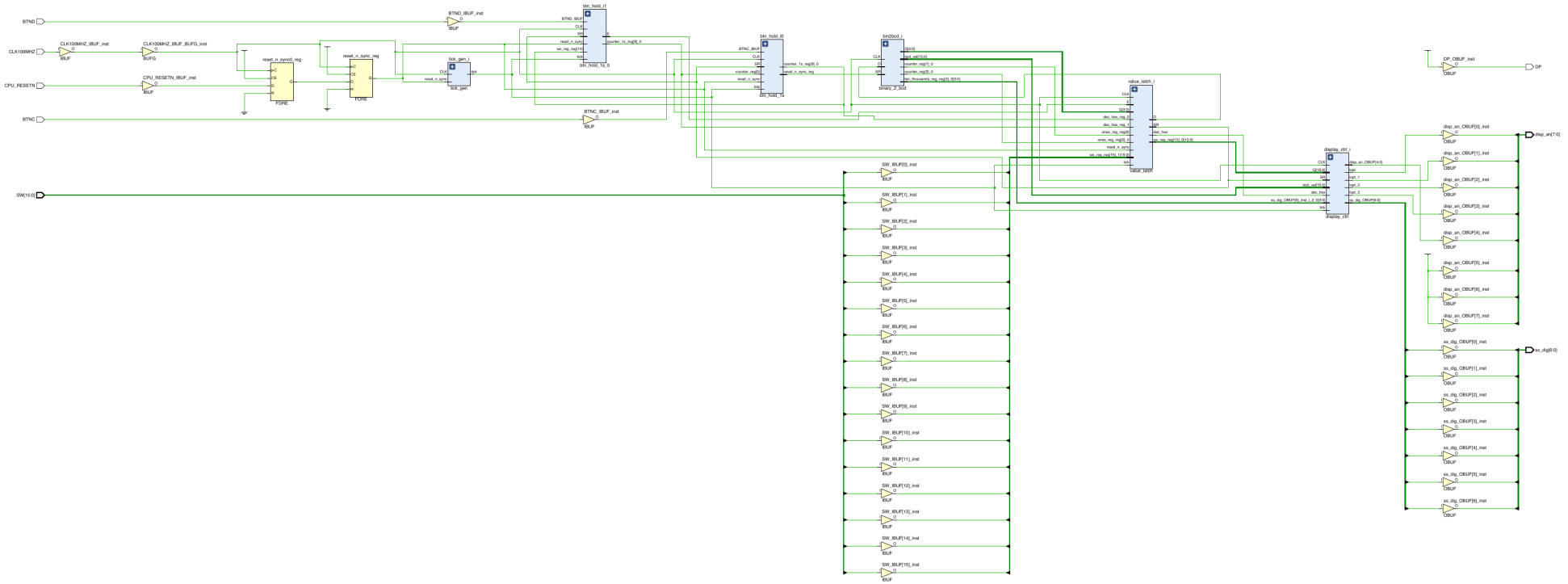


Figure 32: Implemented chip_top (post-route netlist). Per-module implemented schematics are in 4_Implementation_Reports.

7. Resource expectations and usage

This section is done in two stages. Stage 1 is a prediction made from the RTL alone, before any utilization report is opened; stage 2 is the measured usage from the report files, and the comparison between the two. The numbers in stage 2 are taken from utilization_synth.rpt, utilization_placed.rpt and the per-module out-of-context utilization_<module>.rpt files (report_utilization counts slice LUTs after LUT combining; the build's text summary counts LUT primitives, which is why it prints 110 instead of 83/80).

7.1 Stage 1 – expectations from the RTL (filled in before looking at the reports)

Flip-flops are counted exactly from the register declarations; LUTs are an estimate per module from the adders, comparators, multiplexers and decoders in the code. (Worksheet: 5_Resource_Usage/1_resource_expectations_worksheet_FILL_IN_FIRST.md.)

Module (instances)	Registers in the RTL	Expected FFs	Logic needing LUTs	Expected LUTs
chip_top (1)		____		____
tick_gen (1)	counter: ⌈log2(50000)⌉ = ____ bits	____		____
btn_hold_1s (2)	synchronizer ____ + counter ⌈log2(2001)⌉ = ____ bits	each ____ / both ____		each ____
value_latch (1)		____		____
binary_2_bcd (1)		____		____
display_ctrl (1) incl. seven_segment		____		____
seven_segment alone		____		____
Total / full design		____	more or less than the sum? why?	____

Other resources	Expected	Reason
Block RAM / DSP48 / latches	____	
I/O pins (ports of chip_top)	____	
Clock buffers, clock domains	____	
Timing at 100 MHz (longest path, margin)	____	
Power (static vs dynamic)	____	

7.2 Stage 2 – usage measured from the reports

Module (instances)	FFs (OOC .rpt)	Slice LUTs (OOC .rpt)	Other primitives (OOC)
chip_top top level (1)	2	0	1 BUFG, 20 IBUF, 16 OBUF
tick_gen (1)	16	7	4 CARRY4
btn_hold_1s (2)	13 each (26)	15 each (30)	–
value_latch (1)	17	3	–
binary_2_bcd (1)	25	17	2 MUXF7, 1 MUXF8
display_ctrl (1) incl. seven_segment	5	37	–
seven_segment alone	0	7	–
Sum of modules	91	94	
Full design after synthesis (utilization_synth.rpt)	91 (87 FDRE + 4 FDSE)	83 (110 LUT primitives)	4 CARRY4, 1 MUXF7
Full design after placement (utilization_placed.rpt)	95 (88 FDRE + 7 FDSE)	80 (110 LUT primitives)	4 CARRY4, 1 MUXF7

Resource (implemented design)	Used	Available	Utilization
Slice LUTs (all as logic)	80	63,400	0.13 %
Slice registers (flip-flops, 0 latches)	95	126,800	0.07 %
Slices (27 SLICEL + 8 SLICEM)	35	15,850	0.22 %

Resource (implemented design)	Used	Available	Utilization
CARRY4	4	15,850	0.03 %
F7 muxes	1	31,700	< 0.01 %
Block RAM / DSP48	0 / 0	135 / 240	0 %
Bonded IOB (20 in + 16 out)	36	210	17.14 %
BUFGCTRL (CLK100MHZ only, no MMCM/PLL)	1	32	3.13 %

Primitives (placed): FDRE 88, FDSE 7, LUT6 37, LUT4 37, LUT5 16, LUT2 11, LUT3 6, LUT1 3, CARRY4 4, MUXF7 1, IBUF 20, OBUF 16, BUFG 1. Vivado's RTL statistics list one 16-bit, two 11-bit, two 5-bit, five 4-bit and seven 1-bit registers (91 bits), 0 latches.

Two facts from the reports that the comparison must account for: (a) implementation has 4 flip-flops more than synthesis – a comparison of the synthesized and routed checkpoints shows them to be display_ctrl_i/ring_reg_reg[0..3]_lopt_replica, created by opt_design to split the fanout of the scan-ring bits between the anode output buffers and the digit multiplexer (a fanout optimization, no functional change; it is also why FDSE grows from 4 to 7); (b) the integrated design uses fewer slice LUTs (80) than the sum of the modules synthesized alone (94) because constants are propagated (DP, AN7..AN5), the two button detectors share the tick and reset terms, and the display multiplexer sees registers instead of free ports.

7.3 Do the used resources meet the expectations?

____ To be written after completing stage 1: compare each row of 7.1 with 7.2 (flip-flops should match exactly; explain any module whose LUT count is above or below the estimate, in particular display_ctrl and binary_2_bcd), state whether BRAM/DSP/latches/extra clock buffers appeared, and conclude on the overall utilization (0.22 % of the slices, 17 % of the I/O) and on the timing margin. 5_Resource_Usage/2_resource_usage_check_yourself.md holds the full worked comparison to check against. ____

8. Bitstream and board test

Generate Bitstream on impl_chip_top completed with 0 errors, 0 critical warnings and 0 warnings; the bitstream is lab_3/chip_top.bit inside lab_3.zip (also 6_Bitstream/chip_top.bit). The board was programmed through JTAG and the controller was checked on the Nexys A7-100T: after reset the display shows 0 on the rightmost digit; setting the switches and holding BTNC for one second shows the value in decimal on up to 5 digits; holding BTND shows it in hexadecimal on 4 digits; short presses change nothing; the CPU reset button clears the display to 0. The delivered bitstream was regenerated from the final sources and its configuration data is byte-identical to the one tested on the board (only the header timestamp differs).

9. Conclusions

- A complete 7-segment controller with decimal/hex presentation, 1-second button latching and 400 Hz flicker-free refresh was designed in 7 small synchronous modules on one 100 MHz clock.
- Every module and the whole system are covered by self-checking testbenches: 798 checks pass; the waveforms show every requirement (latching, hold time, single pulse, BCD conversion, 5-digit scan, 5th digit only in decimal).
- Lint, elaborated-design DRC/methodology, synthesis, implementation and bitstream generation are clean; the only messages are the expected constant-output warnings and the informational I/O-delay methodology checks.
- Timing is met with 5.3 ns of margin at 100 MHz; the implemented design uses 80 slice LUTs / 95 flip-flops / 35 slices (utilization_placed.rpt) – see section 7 for the comparison with the stage-1 expectation.

Appendix A – Deliverables

Path	Contents
lab_3.zip	lab_3/src/*.v (7 design files), lab_3/sim/*.v (7 testbenches), lab_3/constraints/NexysA7-100T_Master.xdc, lab_3/lab_3.xpr (Vivado 2025.2 project, paths relative to the zip), lab_3/chip_top.bit
Lab3_Report_Seven_Segment_Controller .docx / .pdf	this report
1_Simulation_Waveforms/	23 annotated waveform figures (overview + zooms per testbench), simulation_results_summary.txt, simulation_logs/<tb>_xsim.log
2_Lint_and_RTL_Schematics/	Vivado lint report + summary, Verilator lint per module, elaborated DRC / methodology / noise reports, RTL schematics (png + pdf) of all 7 modules
3_Synthesis_Reports/	synthesis_run.log, summary, utilization (flat + hierarchical), timing, DRC, methodology, power, clock, noise reports, synthesized schematics
4_Implementation_Reports/	implementation_run.log, summary, utilization placed/hierarchical, timing routed, DRC, methodology, route status, I/O, power, control sets, clock utilization, bus skew, implemented schematics
5_Resource_Usage/	1_resource_expectations_worksheet_FILL_IN_FIRST.md (stage 1, blank), 2_resource_usage_check_yourself.md (stage 2, worked comparison), utilization reports of the full design, per-module out-of-context synthesis (utilization, longest path, schematic, module_comparison_summary.txt)
6_Bitstream/	chip_top.bit and the bitstream run summary
vivado_flow_SUMMARY.txt	one-page summary of every flow step as produced by the build (its 'Resources' lines are annotated with the slice-LUT counts from the .rpt files)

Appendix B – Source listing

src/chip_top.v

```
`timescale 1ns/1ps
module chip_top #(
    parameter MAX_TICK = 50000,
    parameter MAX_COUNT = 2000
 ) (
    input CLK100MHZ, CPU_RESETN,
    input [15:0] SW,
    input BTNC, BTND,
    output DP,
    output [6:0] ss_dig,
    output [7:0] disp_an
);
    wire tick;
    wire pressed_c, pressed_d;

    wire start;
    wire [19:0] bcd_val;

    wire dec_hex;
    wire [15:0] sw_reg;

    reg reset_n_sync = 1'b0, reset_n_sync0 = 1'b0;
    always @(posedge CLK100MHZ) begin
        reset_n_sync0 <= CPU_RESETN;
        reset_n_sync <= reset_n_sync0;
    end

    tick_gen #(.MAX_TICK(MAX_TICK)) tick_gen_i (CLK100MHZ, reset_n_sync, tick);

    // Center buton
    btn_hold_1s #(.MAX_COUNT(MAX_COUNT)) btn_hold_i0 (CLK100MHZ, reset_n_sync, BTNC, tick, pressed_c);
    // Down buton
    btn_hold_1s #(.MAX_COUNT(MAX_COUNT)) btn_hold_i1 (CLK100MHZ, reset_n_sync, BTND, tick, pressed_d);

    value_latch value_latch_i (CLK100MHZ, reset_n_sync, pressed_c, pressed_d, SW, dec_hex, sw_reg);

    assign start = pressed_c | pressed_d;

    binary_2_bcd bin2bcd_i (CLK100MHZ, reset_n_sync, sw_reg, start, bcd_val);

    display_ctrl display_ctrl_i (CLK100MHZ, reset_n_sync, tick, dec_hex, sw_reg, bcd_val, disp_an, ss_dig, DP);

endmodule
```

src/tick_gen.v

```
`timescale 1ns/1ps
module tick_gen #(parameter MAX_TICK = 50000) (
    input clk, reset_n,
    output tick
);
    localparam TICK_BITS = $clog2(MAX_TICK);
    localparam [TICK_BITS-1:0] MAX_CNT = MAX_TICK -1;
    localparam [TICK_BITS-1:0] ONE = 1;

    reg  [TICK_BITS-1:0] counter;

    assign tick = counter == MAX_CNT;

    always @(posedge clk) begin
        if (!reset_n | tick) counter <=  {TICK_BITS{1'b0}};
        else counter <= counter + ONE;
    end


endmodule
```

src/btn_hold_1s.v

```
`timescale 1ns/1ps
module btn_hold_1s #(parameter MAX_COUNT = 2000) (
    input clk, reset_n,
    input click,
    input tick,
    output pressed_1s
);

    // Need a synchronizer for click because is is async coming into a synch system
    reg click_sync, click_sync0;
    always @(posedge clk) begin
        if (!reset_n) {click_sync, click_sync0} <= 2'b00;
        else begin
            click_sync0 <= click;
            click_sync <= click_sync0;
        end
    end

    // Need to gen a 1s tick counter from the 400 tick gen
    // 2kHz * 1/1Hz = 2000 -> Need to count 2000 ticks of tick. need log2(2000+1) = 10.96 -> reg [10:0] (11 bits)
    // we need 2001 because their is no wrap around so we dont count it, and need to compinsate
    localparam MAX_COUNT_BITS = $clog2(MAX_COUNT + 1);
    localparam [MAX_COUNT_BITS-1:0] MAX_CNT_M1 = MAX_COUNT - 1;
    localparam [MAX_COUNT_BITS-1:0] MAX_CNT = MAX_COUNT;
    localparam [MAX_COUNT_BITS-1:0] ONE = 1;

    reg [MAX_COUNT_BITS-1:0] counter_1s;

    always @(posedge clk) begin
        if (!reset_n | !click_sync) counter_1s <= {MAX_COUNT_BITS{1'b0}};
        else if (tick) begin
            if (counter_1s == MAX_CNT) counter_1s <= counter_1s;
            else counter_1s <= counter_1s + ONE;
        end
    end

    assign pressed_1s = click_sync & tick & (counter_1s == MAX_CNT_M1);


endmodule
```

src/value_latch.v

```
`timescale 1ns/1ps
module value_latch (
    input clk, reset_n,
    input pressed_c, pressed_d,
    input [15:0] sw,
    output reg dec_hex,     // When 1 decimal is pressed, when 0 hex is pressed, reseted to hex view
    output reg [15:0] sw_reg
);
    always @(posedge clk) begin
        if (!reset_n | pressed_d) dec_hex <= 1'b0;
        else if (pressed_c) dec_hex <= 1'b1;
    end

    always @(posedge clk) begin
```

```
        if (!reset_n) sw_reg <= 16'b0;
        else if (pressed_c | pressed_d) sw_reg <= sw;
    end

endmodule
```

src/binary_2_bcd.v

```
`timescale 1ns/1ps
module binary_2_bcd (
    input clk, reset_n,
    input [15:0] sw,
    input start,
    output [19:0] bcd
);

    // Double Dabble algorithim.
    // If a part (ones/tens/hundreds..) >= 5 then we need to add 3 to that number
    // No need to check ten_thousands because max digit is 6, so before last shift is was 3 (0011) -> (0110) so no need to
    // one big shift regesiter between the added parts
    // counter to make sure we do 16 shifts and thats the end.
    // can also add edge case where sw == 16'b0 and reduce calculation time. but wont see a difference on screen because ca
    wire [3:0] ones, tens, hundreds, thousands;
    reg [3:0] ones_reg, tens_reg, hundreds_reg, thousands_reg, ten_thousands_reg;
    reg [4:0] counter;

    assign ones = (ones_reg >= 4'd5) ? ones_reg + 4'd3 : ones_reg;
    assign tens = (tens_reg >= 4'd5) ? tens_reg + 4'd3 : tens_reg;
    assign hundreds = (hundreds_reg >= 4'd5) ? hundreds_reg + 4'd3 : hundreds_reg;
    assign thousands = (thousands_reg >= 4'd5) ? thousands_reg + 4'd3 : thousands_reg;

    wire shift_16 = counter == 5'd16;

    always @(posedge clk) begin
        if (!reset_n | start) counter <= 5'b0;
        else begin
            if (shift_16) counter <= counter;
            else counter <= counter + 5'd1;
        end
    end

    always @(posedge clk) begin
        if (!reset_n | start) begin
            {ten_thousands_reg, thousands_reg, hundreds_reg, tens_reg, ones_reg} <= 20'b0;
        end
        else if (!shift_16) begin
            ones_reg <= {ones[2:0], sw[15-counter]};
            tens_reg <= {tens[2:0], ones[3]};
            hundreds_reg <= {hundreds[2:0], tens[3]};
            thousands_reg <= {thousands[2:0], hundreds[3]};
            ten_thousands_reg <= {ten_thousands_reg[2:0], thousands[3]};
        end
    end

    assign bcd = {ten_thousands_reg, thousands_reg, hundreds_reg, tens_reg, ones_reg};
endmodule
```

src/display_ctrl.v

```
`timescale 1ns/1ps
module display_ctrl (
    input clk, reset_n,
    input tick, // 2kHz tick
    input dec_hex,
    input [15:0] hex_val,
    input [19:0] bcd_val,
    output [7:0] display_an,
    output [6:0] ss_dig,
    output dp
);

    reg [4:0] ring_reg;

    always @(posedge clk) begin
        if (!reset_n) ring_reg <= 5'b11110;
        else if (tick) ring_reg <= {ring_reg[3:0], ring_reg[4]};
    end

    assign display_an = {3'b111, !dec_hex | ring_reg[4], ring_reg[3:0]}; // needed to be zero when dec_hex is 1 and ring[4]

    wire [19:0] val_to_show = (dec_hex) ? bcd_val : {4'b0, hex_val};

    reg [3:0] dig_in;
```

```
    always @(*) begin
        case (ring_reg)
            5'b11110: dig_in = val_to_show[3:0];
            5'b11101: dig_in = val_to_show[7:4];
            5'b11011: dig_in = val_to_show[11:8];
            5'b10111: dig_in = val_to_show[15:12];
            5'b01111: dig_in = val_to_show[19:16];
            default: dig_in = val_to_show[3:0];
        endcase
    end

    seven_segment ss_i (dig_in, ss_dig);

    assign dp = 1'b1;

endmodule
```

src/seven_segment.v

```
`timescale 1ns/1ps
module seven_segment (
    input [3:0] dig_in,
    output reg [6:0] ss_dig
);

    always @(*) begin
        case (dig_in)
            4'h0: ss_dig = 7'b0000001;
            4'h1: ss_dig = 7'b1001111;
            4'h2: ss_dig = 7'b0010010;
            4'h3: ss_dig = 7'b0000110;
            4'h4: ss_dig = 7'b1001100;
            4'h5: ss_dig = 7'b0100100;
            4'h6: ss_dig = 7'b0100000;
            4'h7: ss_dig = 7'b0001111;
            4'h8: ss_dig = 7'b0000000;
            4'h9: ss_dig = 7'b0000100;
            4'hA: ss_dig = 7'b0001000;
            4'hB: ss_dig = 7'b1100000;
            4'hC: ss_dig = 7'b0110001;
            4'hD: ss_dig = 7'b1000010;
            4'hE: ss_dig = 7'b0110000;
            4'hF: ss_dig = 7'b0111000;
            default: ss_dig = 7'b1111111;
        endcase
    end

endmodule
```